Robot Teleoperated Hand

Beverley, Kenji, Joyce, Siyi

Version 0.4

History

Revision	Changes	Author(s)	Date
0.1	Add Understanding the Problem and Motivation in Requirement Specification Section	Kenji	2025/09/24
0.2	Use Cases and Detailed Scenarios in Requirement Specification Section	Beverley	2025/09/25
0.3	Functional specification in Requirement Specification Section	Joyce	2025/09/25
0.4	Non-functional specification in Requirement Specification Section	Siyi	2025/09/25
1.1	Outlined the concept design	All	2025/10/25
1.2	Updated wearable device subsystem, system description, concept of operation, and software architecture diagram	Beverley	2025/10/28
1.3	Updated the communication subsystem, hardware/software deployment, and the first two concept trade studies	Joyce	2025/10/28
1.4	Updated the subsystem description of the robotic hand, added the sequence diagrams and scenarios for two modes.	Siyi	2025/10/28
1.5	Updated trade studies for finger actuation and hand position sensing. Planned steps for prototyping. Design relative to requirements	Kenji	2025/10/28
1.6	Added references to functional and non-functional matrix for traceability between decisions and requirements	Kenji	2025/12/11
1.7	Refined concept of operations	Beverly	2025/12/11
2.1	Distinguish between sections	All	2025/11/12
2.2	Updated the requirements validation testings	Beverley	2025/11/13
2.3	Updated the functional verification testings and made plan to edit functional requirements	Joyce	2025/11/13

2.4	Updated the performance testing	Siyi	2025/11/13
2.5	Updated failure modes sections	Kenji	2025/11/13
2.6	Added references to requirements, added a traceability matrix	Team	2025/12/11
3.1	Outlined	Team	2025/12/11
3.2	Hardware fabrication, CAD, Mechanical assembly, Trade studies	Kenji	2025/12/11
3.3	Provide software module and interaction description and diagrams (object-oriented or structured design), Fabrication process for software, and Described fault recovery and degraded modes for software	Bev	2025/12/11
3.4	Sequence diagrams, state chart diagrams	Siyi	2025/12/11
3.5	Electrical Wiring, Fault recovery, parts list, fabrication process for electronics	Joyce	2025/12/11
4.4	Management plan	Kenji, Joyce, Siyi, Bev	2026/1/26

Contents

Requirement Specification

1. Understanding the Problem

There are many environments that humans are required to go to that are incredibly dangerous. Robots have already begun to do very dangerous tasks however, many tasks are simply too complicated for robots to do. We hope to change this by creating hand-like robotic manipulators that can do tasks that require very fine motor skills. Some examples of dangerous environments humans still have to enter include:

Low oxygen environments:
In enclosed spaces such as ship fuel tanks or caves oxygen in these areas react with the surrounding creating an environment that contains little to no oxygen. Humans typically have to enter these spaces with supplemental oxygen, if anything goes wrong or the space is entered without knowing that there is reduced oxygen the person can die.

High Voltage Electronics:
In areas with high voltage electronics, there are often controls and fine electronics that need to be maintained, this is normally done by a human. Currently humans do this wearing large flash suits that in the case of an arc flash may save them. It is also common for another person to stand behind them with a large hook so that if they were to get shocked, they could be removed from the situation. (https://rauckmanutility.com/products/insulated-rescue-hook/)

High Risk of falling:
There are places such as wind turbines and cell towers that though not often do need maintenance. Wind turbine gears and generators need occasional maintenance. Cell towers and similar structures require having lights so that planes can see them that occasionally need to be replaced. This requires humans to go hundreds of feet up into the air to complete fine motor tasks. If robots could do the maintenance, risk to humans would be reduced.

Lab/mechanic shops:
In spaces such as labs and mechanic shops people occasionally have to do things that are quite dangerous. A chemist may need to be in the presence of a reaction that produces dangerous fumes while adjusting a separatory funnel. A mechanic may need to remove compressed suspension springs from a car. These are both tasks if done by a teleoperated robot would eliminate any risk from the person.

Radioactive Environments:

After the Chernobyl, Fukushima, and three mile island incidents, humans needed to clean up. Though it would be optimal for this not to happen, if it were to happen robots could clean up radioactive waste.

Biohazards:
In the COVID19 pandemic, though the virus was very contagious, sick patients still needed to be treated. Employing robots to take care of sick patients will reduce the risk of viruses spreading

In all of these scenarios, the core challenge is achieving adequate telepresence: operators need real time sensory input such as tactile feedback for object contact and force modulation for delicate tasks.

2. Motivation

Adopting a robotic hand with sensor feedback as a solution is driven by the possibility of enhancing mainly safety but also efficiency and accessibility in ways where current technologies fall short. The major motivation for a solution in this area is removing humans for direct exposure to life threatening environments. This will prevent fatalities, injuries, and other health issues. For instance instead of suiting up for a high voltage repair on machinery an operator could control a robot to do the same thing from the safety of somewhere else.

Beyond safety this approach would also increase productivity. A single operator instead of traveling from location to location could oversee robots across distant locations. A single operator in one location instead of traveling to 5 different locations over 5 days could use the local maintenance robot and get all 5 done in the same day. On top of that, operators could specialize more, this would allow a very specialized operator to help with any problem on site rather than whoever happened to be nearby.

Economically, robotic solutions could reduce costs associated with human risk management, such as training, insurance, ppe, downtime, and being extra safe because humans are irreplaceable. As more robots perform the tasks, they will also generate a lot of data that in itself is very valuable. Potentially someday this training data could be used to train robots.

Assumptions made:
To scope our project effectively we made some assumptions about the problem domain and the robotic hand's implementation:

- Object manipulability: The objects targeted for manipulation are relatively easy to move, with reasonable mass ($\leq$1 kg) and a maximum grasp width of $\leq$7 cm. This is because we want to focus on the fine manipulation aspect rather than power

- Gripper design: the robotic has will feature at least two fingers for basic pinching and grasping. We are not targeting anthropomorphism but rather manipulation.
- Connectivity: the robot will be able to operate on a reliable low latency communication between the operator and the robot.
- Power: For testing purposes or robot will be power with a power supply rather than a battery because managing batteries and charging would only take away from attention that would be needed for the rest of the project
- Operator skill level: users are assumed to have basic training in teleoperation with the systems hopefully more intuitive feedback would be reasonably easy for people to use

These assumptions allow us to focus on a prototype that delivers on the aspects that are important to the robot and the spirit of our goals.

3. Use Cases and Detailed Scenarios

Primary Actors

- Human Operator wearing hand device
    - Directly controls robotic hand
- Robotic Hand
    - Executes operator's motions
    - Collects tactile/force/pressure data from the environment
    - Sends data back to haptic system

Secondary Actors

- Wearable device / Haptic Device
    - Communicates with robotic hand
    - Provides force/pressure feedback simulating touch
- Sensed Object / Environment
    - Object robotic hand interacts with or touches

Scenarios

1. **Grabbing a Grape**

   Actors:
   - Operator, Robotic Hand, Wearable device

   Environment:
   - Grape (Sensed Object)

   Goal:
   - The operator wants to pick up and hold a grape without crushing it.

   Precondition:
   - Grape is resting on an open flat surface and not already held by hand
   - Wearable device is on Operator's hand

   Main Flow:
   1. Operator moves their hand in the direction of the grape

2. Robotic hand moves towards grape following command from operator
3. Robotic hand makes contact with the grape, detects pressure/tactile data, and sends data to the wearable device
4. Wearable device simulates touch to the operator
5. Operator adjusts grip based on touch and robotic hand securely holds grape with appropriate force

Alternatives:

- Object is a marshmallow (soft)
    - Robotic hand applies less force, wearable device conveys softness to operator, and operator adjusts grip accordingly
- Object is a steel brick (hard)
    - Operator applies more force, wearable device conveys hardness of surface to operator, and operator adjusts grip accordingly

Postcondition:

- Success: Grape is securely held without damage
- Failure: Grape slips or is crushed

2. Pressing a Button

Actors:

- Operator, Robotic Hand, Wearable device

Environment:

- Button (Sensed Object)

Goal:

- The operator wants to press a button.

Precondition:

- The button is unpressed and will perform an action when pressed
- Wearable device is on operator's hand

Main Flow:

1. Operator moves their hand toward button
2. Robotic hand moves towards button following operator's command
3. Robotic hand makes contact with the button, sensors detect initial contact, and send touch feedback to wearable device
4. Operator applies additional force to press the button
5. Robotic hand presses button with enough force to trigger it's function
6. Robotic hand detects press and sends press feedback to wearable device
7. Operator perceives confirmation that the button has been pressed

Alternatives:

- If the robotic hand touches a non-pressable surface (like a wall or table):
    - Sensors detect contact and send touch feedback to wearable device
    - No press feedback is given

- Operator realizes the surface is not pressable

Postcondition:

- Success: Button is pressed, and operator is given press feedback
- Failure: Button is not pressed

3. **Feeling Surface Roughness**

Actors:

- Operator, Robotic Hand, Wearable device

Environment:

- Sandpaper (an object with a textured surface)

Goal:

- The operator wants to perceive the roughness or smoothness of a surface.

Preconditions:

- Sandpaper is stationary and does not move when robotic hand moves along its surface
- Wearable device is on operator's hand

Main Flow:

1. Operator moves their hand toward the sandpaper
2. Robotic hand follows operator's motion and makes contact with the surface of the sandpaper
3. Robotic hand detects texture features of sandpaper's surface and sends data to wearable device
4. Wearable device simulates the rough tactile sensation of the sandpaper's surface
5. Operator moves their hand along the surface to feel texture's variations
6. Operator perceives various roughness of the surface from the wearable device

Alternatives:

- If the surface is uniformly smooth or cannot be sensed:
  - Wearable device provides very minimal or no texture feedback
  - Operator recognizes the surface is smooth or featureless

Postconditions:

- Operator perceives accurate roughness/smoothness of the surface

4. **Lifting a Container with Shifting Contents**

Actors:

- Operator, Robotic Hand, Wearable device

Environment:

- Container with movable internal contents

Goal:

- Operator wants to lift and move around a container while perceiving changes in weight and internal movement

Preconditions:

- Internal contents of container are free to shift without falling out
- Wearable device is on operator's hand

Main Flow:

1. Operator moves their hand toward the container
2. Robotic hand follows operator's motion and grasps the container securely
3. Robotic hand detects weight of the container and any internal movement of its contents
4. Data is sent to wearable device, which simulates dynamic changes in weight and force to operator
5. Operator lifts and tilts the container, feeling the shifting resistance as the container's contents move
6. Operator adjusts grip and orientation of hand to maintain control of container
7. Robotic hand provides feedback until operator releases container

Alternatives:

- If the container is empty or its contents are in a fixed:
    - Wearable device provides constant weight feedback
    - Operator perceives no internal movement

Postconditions:

- Success: Operator perceives changing weight and internal movement accurately, while maintaining control of the container
- Failure: Operator misjudges changing weight or shifting contents, dropping or destabilizing the container

5. Throwing an Object

Actors:

- Operator, Robotic Hand, Wearable device

Environment:

- Ball (Sensed Object)

Goal:

- The operator wants to pick up and throw a ball, perceiving the weight and release timing

Preconditions:

- The ball is free to be grasped and thrown
- Wearable device is on operator's hand

Main Flow:

1. Operator moves their hand towards the ball
2. Robotic hand follows operator's motion and grasps the ball securely
3. Robotic hand detects weight and grip force, sending data to wearable device
4. Operator moves arm in throwing motion and robotics hand mirrors movement

5. Operator feels increasing resistance from ball's weight and inertia through wearable device
6. At throw point, operator releases grip and the robotic hand releases the ball
7. Wearable device simulates the sudden loss of weight/resistance at release

Postconditions:

- Success: Ball is thrown with controlled release, and operator perceives accurate weight/resistance throughout motion
- Failure: Ball slips or is not released at intended moment

4. Functional Requirements Specification

A. Operator Intent Capture (Priority: high)

1. The system shall capture the operator's arm and hand motions using a wearable device.
2. The system shall detect individual finger motions and intent forces.
3. The system shall support calibration and re-centering between the wearable device and the robotic arm/hand.

B. Motion Mapping & Control (Priority: high)

4. The robotic arm/hand shall mirror the operator's motions in real time.
5. The robotic arm/hand shall, when commanded, move to reproduce the operator's current arm and hand gesture at a controlled, safe speed.
6. The system shall support both control modes: (a) continuous real-time tracking and (b) mimic gesture of the user when commanded.
7. The system shall evaluate whether intended motions are within the robot's capabilities (e.g., speed, reach, or limits) and prevent infeasible or unsafe commands.
8. The robotic hand shall allow continuous control of grip gestures, aperture, and applied force.
9. The robotic hand shall securely hold objects within its designed payload range.

C. Sensing & Feedback (Priority: high)

10. The robotic hand shall detect contact at the fingertips.
11. The robotic hand shall measure grasping forces at the fingertips.
12. The robotic hand shall sense surface texture during contact.
13. The system shall provide the operator with real-time feedback about contact, force, and surface properties.

D. Communications & Interfaces (Priority: high)

14. The system shall support bidirectional communication between the wearable and the robot.

15. The system shall maintain time synchronization between the wearable, the robot, and logged data.
16. The system shall log its internal knowledge of operator intent, robot state, sensor inputs, commanded forces on the robot arm, and feedback signals to the wearable device.
17. The system shall have some ability to display its knowledge in real time, including visualizing user pose, robot pose, and show current and changes in sensor data, commanded forces, and feedback signals.

E. Safety & Fault Handling (Priority: highest)

18. The system shall require safety check procedures before enabling motion.
19. The system shall reject connection requests from unauthorized devices.
20. The system shall monitor the robot's environment (e.g., workspace boundaries) and prevent motions that would cause unsafe contact or collisions.
21. The system shall provide safe fallback behavior for the robotic hand/arm when communication or tracking quality degrades or is lost (e.g., hold position or return to a defined home pose).
22. The system shall provide safe fallback behavior for the wearable device when communication or tracking quality degrades or is lost (e.g., gradually releasing all force and stopping haptic feedback).
23. The system shall include an emergency stop accessible to both the operator and a nearby observer.

5. Nonfunction Requirements

The requirements are ranked from high priority to low priority

A. Performance

1. Control Loop Latency: The end-to-end latency (wearable input → robot actuation → feedback to wearable) shall not exceed 200 ms.

Justification:

According to *A Brief Survey of Telerobotic Time Delay Mitigation* (Frontiers in Robotics and AI, 2020) [3] , operator performance in teleoperation tasks shows little difference between no delay and delays up to **100–250 ms**, with significant degradation typically appearing only beyond ~400 ms.

Breakdown:

- Sensing latency ≤ 20 ms
- Communication latency ≤ 30 ms
- Robot actuation latency ≤ 100 ms
- Feedback rendering latency ≤ 50 ms

2. Pose Tracking Accuracy: The robotic end-effector shall match the operator's commanded pose with position error ≤ 10 mm and orientation error ≤ 5° in free space.

3. Grip Force Range and Resolution: The robotic hand shall exert fingertip forces from 0.2 N to 20 N with resolution ≤ 0.3 N from 0.2 N to 10 N, and resolution < 2N from 10 to 20 N. This will allow the hand to manipulate a range of objects from a grape to a button.

- 0.2 N = enough to sense a grape; 20 N ≈ firm grasp on a small tool

4. Payload Capacity: The robotic hand shall lift and hold objects up to 0.5 kg. (0.5kg is approximately the mass of a 500ml water bottle)

5. Contact Detection: Fingertip contact events shall be detected within 20 ms of threshold crossing.

6. Workspace: The robotic arm shall reach a workspace radius of at least 0.5 m from its base.

7. Feedback Fidelity: Haptic feedback shall reflect contact force changes with update rate ≥ 20 Hz

B. Useability and Availability

The hand and gloves robot system needs to provide a good amount of telepresence to the user. The criteria of a good telepresence are: similar perceived distance to the actual distance between the hand and the objects, users being able to properly feel the exerting pressure and texture, and unnoticeable time lag between the human input and the actuation. Satisfying these conditions also suggest the system is easy to operate without extensive training. Depending on the actual scenario, the availability of our system varies. In general, the robot needs to be constantly available and respond promptly to the user's command.

C. Safety

To ensure the safety of the system, we need to ensure the robot hand will not perform anything harmful to its environment and to itself. Therefore, the system can be shut down at any moment for safety reasons defined by the human and a prescripted supervision system. For every operating scenario, the defined robot configuration space should always be "safe" from anything prohibited by the objective(for instance, 2N grip force will be a forbidden state to reach if the hand needs to grab a grape).

D. Reliability

In terms of reliability, the robot hand needs to be robust in structure since it needs to actuate heavy objects and operate in environments that are potentially subjected to high wind turbulence or under high temperature. In addition, we want the robot to have a five year life cycle,provided no severe damage occurs, in order to minimize the frequency of human exposure to dangerous areas for robot replacement.

E. Cost

Due to the budget limit, we need to keep our overall cost under 2,500. We specify a tentative budget requirement for each of the system's components for better cost management.

Robotic arm: using existing product, estimated budget ≤ **$600**

Robotic hand: including budget for self build mechanical structure, buying motion actuators and sensors, and a small chip for processing ≤ **$1,200**

Wearable device: including budget for self build mechanical structure, buying feedback actuators and position and gesture sensors, and a small chip for processing, cloth for comfort wearing ≤ **$500**

Contingency: for potential damage replacements in building and testing the robot ≤ **$200**

F. Maintainability

The system shall be easy to clean and maintain so that it can quickly recover from any damages and back to operation again. Subsystems like fingers, sensors and feedback devices should be modular and replaceable if damaged.

G. Compatibility

We intend the subsystems to be coordinated by ROS2 since our system requires robust real-time communication and compatibility with other robotic platforms as it can be potentially mounted to a humanoid or a robot arm for future development.

System Concept

1. System Description

1.1 Description of the System in Terms of its Operation

The Teleoperated Robot Hand System enables users wearing a haptic device to remotely control a robotic hand in real time while receiving tactile feedback that represents the robot hand's interaction with its environment. The system is divided into three primary subsystems: the wearable device, the robotic hand, and the communication system.

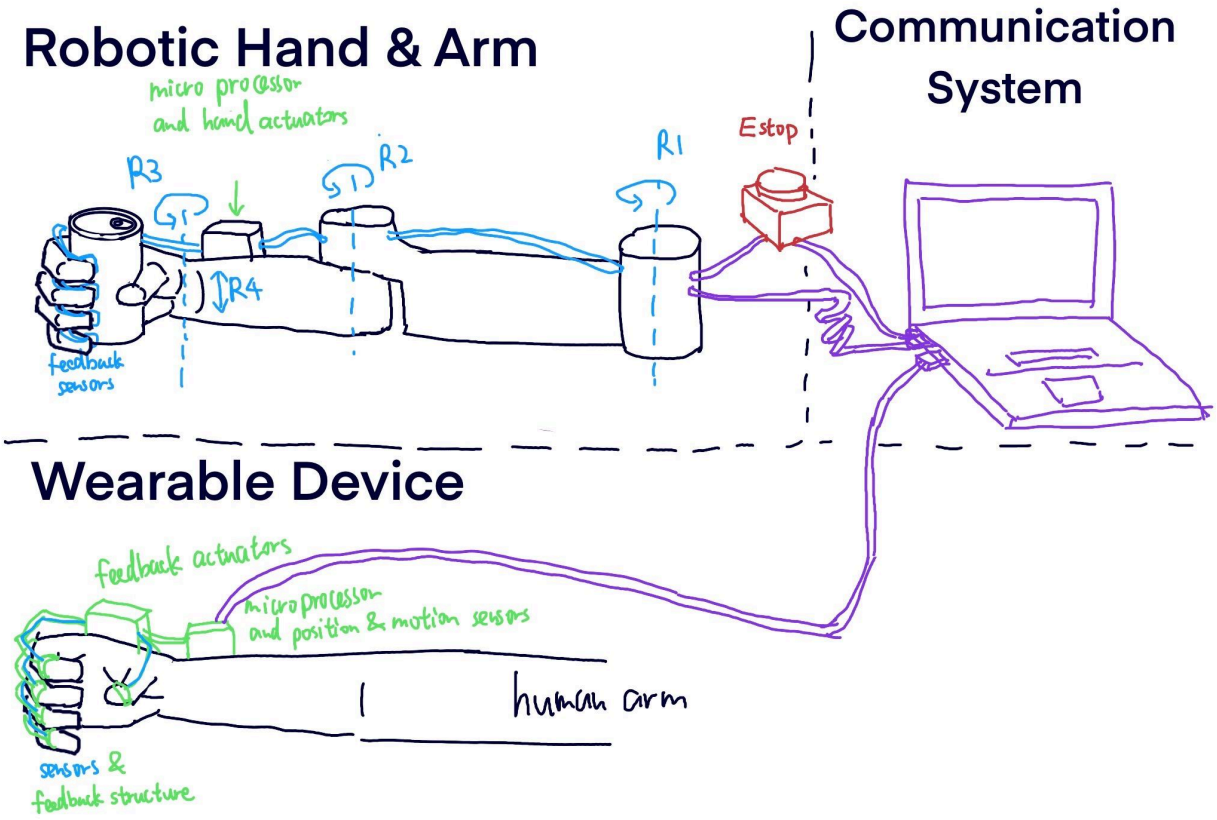
The wearable device measures the user's wrist, hand, and finger motion, as well as grip strength, using flex sensors, IMUs, and force sensors. It transmits this motion data to the communication system while receiving haptic feedback commands based on the robot hand's contact with the environment. The device executes these commands through vibration or resistance cues, allowing the user to feel touch and applied pressure.

The communication system, centered on a computer running ROS 2, manages bidirectional data flow between the wearable device and the robotic hand. It receives motion data from the wearable device, converts it to joint position commands, and sends these to the robotic hand for execution. Simultaneously, it receives tactile and force data from the robot and maps them into haptic feedback signals for the wearable device. This subsystem ensures synchronized timing, safe control, and low-latency teleoperation.

The robotic hand, attached to a multi-DoF robotic arm, executes the operator's intended movements. It receives pose and grip commands from the communication system and mirrors the user's gestures in real time. Its embedded sensors detect contact, force, and motion constraints, sending this data back to the communication system for haptic feedback generation.

Together, these three subsystems form a closed-loop teleoperation architecture that allows intuitive, responsive, and safe human–robot interaction.

1.2. Sketch of the System's Physical Form



2. Concept of Operation

Our teleoperated robot arm–hand system is designed for tabletop manipulation tasks, allowing a user to remotely interact with physical objects via a robotic assembly positioned on a workbench. The robot arm and hand are mounted securely on the table alongside a computer that runs the teleoperation, haptics, perception, and safety-monitoring software.

The operator stands at the workstation wearing a cable-driven force-feedback glove. By moving their hand, the operator naturally controls the robot's hand motions, while haptic feedback conveys contact events occurring within the robot's workspace. This enables the operator to "feel" interactions such as object contact, boundary limits, or resistance during grasping.

The system supports two primary modes of operation, which the user can switch between using their free hand through designated controls on the console or interface:

- Continuous Teleoperation Mode, where the robotic hand continuously mirrors the operator's live hand motion.
- Mimic / Gesture Playback Mode, where the system captures a demonstrated pose from the glove and commands the robotic hand to reproduce and hold that configuration.

To support safe and transparent operation, the workstation computer displays detailed system state information, including robot joint states, live tracking indicators, responsiveness metrics,

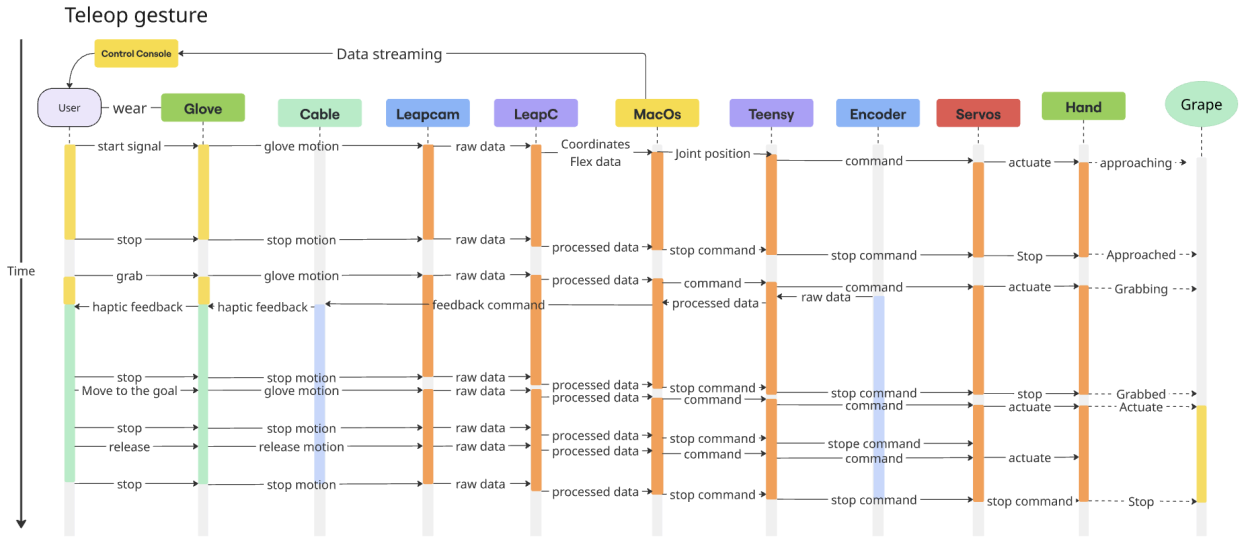
confidence scores from sensing modules, and visualizations of the robot's movements. Safety, fault detection, and recovery panels allow both the operator and supervisor to monitor the system and intervene if needed.

3. Scenarios and Sequence Diagrams

For both modes, the system can do a variety of jobs like grabbing an arbitrary object, lifting an object (potentially with content), mimicking gestures and moving the hand according to the user's hand motion, etc.. We will present two scenarios to better understand the workflow of the robotic system.

A. Realtime teleoperation:

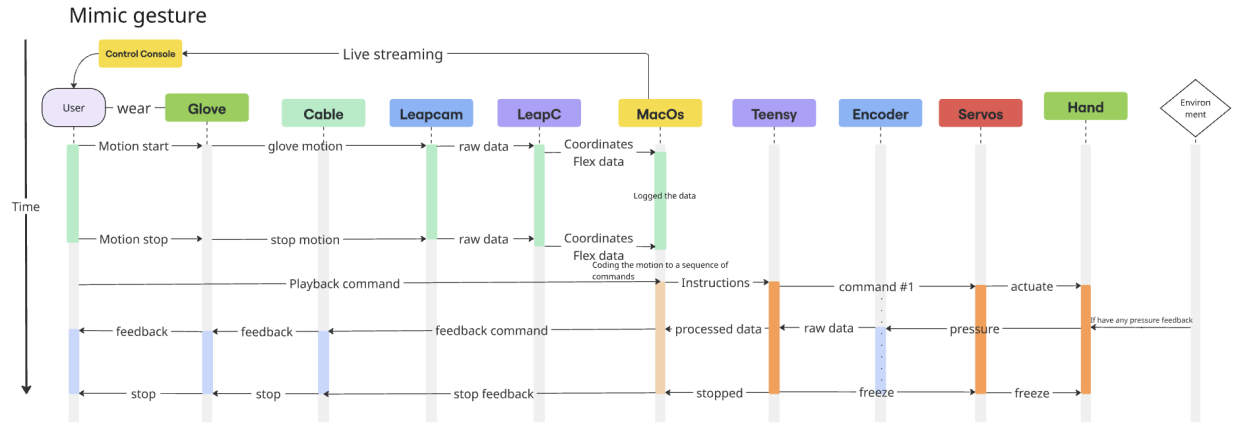
One scenario for realtime teleoperation is grabbing an object and putting it to a goal as summarized in the sequence diagram below. The diagram assumes the system is already powered up(as marked by the grey bars) and constantly listening to the motion sent from the user's glove. The user will also receive a video streaming from the camera, so they know the surrounding environment with respect to the robotic hand. The orange bars on the diagram indicate each subsystem is actively processing the motion data and giving command to the robotic hand. The blue bars represent the active pressure feedback once the hand is in contact with the target object. We didn't explicitly show the time latency between communications, but we expect the time delay to be less than 200 ms.



B. Playback:

A simple scenario for playback is recording and then performing the hand movement of the user. The user will first provide a start signal to notify the computer of entering the playback mode, and the computer will record the subsequent motion of the glove. The recording will stop once

the computer receives a stop signal from the user. The computer will then package the motion to a sequence of instructions(command #) to be sent to the processor for playback. During the playback, any interesting sensory information received from the environment will be transmitted back and sensed by the human hand. Once the last command is performed, the hand will freeze into its last configuration and send a signal back to the computer to notify the end of the playback mode.



4. Subsystem Descriptions

The prosthetic teleoperated hand system can be broken into three major subsystems: the robotic hand attached with a robotic arm, the communication system (as represented as the computer in the above sequence diagram), and the wearable device (glove). We will elaborate the individual components that constitute each subsystem in this section.

A. Robotic Hand System

The **robotic hand system (attached to an arm)** is the robot manipulator teleoperated by the wearable device through the communication system. It consists of an actuation system for movements of hand orientation, position and gesture, a sensor system for collecting pressure and position feedback, a microprocessor for collecting and processing the feedback, and a power source to power the whole robot.

A1. Actuation System:

This system is responsible for controlling the motion of each finger, and the overall mechanical structure of the hand and arm system. We will conduct trade studies and make prototypes in order to finalize the actuation component. Since the actuation system is directly related to the physical robot, the physical structure will be considered as part of the actuation subsystem.

A2. Sensor System:

The system will collect information of the robot hand position and orientation and the pressure experienced on the fingers when grabbing an object. Therefore, we expect to have some pressure sensors built on each finger of the robot, and sensors, like IMU & encoder, to be attached on the hand. There will also be a camera mounted near the hand(or potentially on the hand) to give visual feedback to the user.

A3. Microprocessor

The system serves as the subsystem's controlling center that processes the data sent from the sensors, transmits the packaged data to the computer, receives the command from the computer, and executes them by controlling the actuation system. Therefore, we expected to have a microprocessor (e.g. an arduino chip) mounted on the hand.

A4. Power System

The system will provide power to the entire robot hand and arm, and allocate different voltages according to the need of the actuators and sensors.

B. Communication System

The communication system is centered on a computer that acts as the intermediary between the wearable device and the robotic hand. It manages both the control and feedback pathways, ensuring smooth, real-time translation of user motion into robotic action and robotic sensing into user feedback. The computer connects wirelessly to the wearable device and uses a wired or wireless interface to communicate with the robotic hand's microcontroller. It runs software modules that coordinate data reception, processing, and command transmission with minimal latency.

B1. Control System:

The control component of the communication system interprets motion data received from the wearable device. Sensor readings of hand orientation and position are translated into corresponding joint angle targets and grip commands for the robotic hand. Sensor readings of finger flex angles and applied forces are translated into corresponding motion and grip force commands for the robotic hand. A control algorithm processes these values and converts them into motor actuation commands that are sent to the robotic hand's microprocessor.

B2. Feedback System:

The feedback component works in the opposite direction. It collects tactile and pressure data from sensors on the robotic hand and transmits this information back to the computer. The software processes these signals, mapping them to corresponding feedback patterns (e.g., vibration or force feedback) for the wearable device. This enables the operator to perceive the contact forces experienced by the robotic hand.

B3. Software Function:
The communication software is organized into modules responsible for sensor data acquisition, signal mapping, and command generation. Data from both devices are synchronized, filtered for noise, and transmitted at a consistent update rate to maintain real-time response. The program ensures that motion and feedback signals remain time-aligned and stable, even with minor transmission delays or packet loss. Safety checks are built in to prevent unintentional motion if invalid or missing data are detected.

B4. User Interface and Data Logging:
A simple user interface on the computer provides live visualization of both systems' data—showing robotic joint angles, contact pressure, and network status. It allows users to calibrate sensor sensitivity, adjust motion scaling, or observe haptic response strength. Meanwhile, a background logging process records communication data streams, motor commands, and feedback signals for later review. This supports debugging, performance analysis, and further refinement of control parameters.

B5. Communication Links
The system uses two wired communication links, each designed for fast, stable, and deterministic data exchange. In the intended real-world application, two computers will be used: one connected to the wearable device and another connected to the robotic hand and arm. These computers will communicate with each other over the Internet, enabling remote teleoperation across physical distances. However, for demonstration and testing purposes, the current setup employs a single computer to manage both subsystems locally. This simplifies integration and minimizes the impact of Internet latency during development and system validation.

- **Computer ↔ Wearable Device:**
  A USB or serial connection provides high-rate bidirectional communication and power to the wearable device. Sensor data—including flex sensor readings, IMU orientation, and applied force—are streamed to the computer at a fixed frequency. In return, the computer sends haptic feedback commands that drive the wearable actuators. The wired link ensures minimal latency and prevents signal dropouts during prolonged operation.
- **Computer ↔ Robotic Hand and Arm:**
  A separate wired interface (such as USB or CAN) connects the robotic hand and arm microcontroller to the computer. This connection transmits motion commands, grip force targets, and control mode settings from the computer to the robot, while sending back real-time feedback including motor encoder values, currents, and fingertip pressure readings. The use of a wired connection provides consistent timing and eliminates the risk of wireless interference, which is particularly important for safe and stable teleoperation.

C. Wearable device

The wearable device captures the user's hand and finger movements and sends motion data to the communication system. It also provides haptic feedback based on the robot's hands interaction with the environment allowing the user to feel touch or resistance during operation.

C1. Feedback system:
The feedback system simulates what the robotics hand feels by either creating a sense of touch or resistance helping the user perceive how strongly the robot hand is gripping or when it comes in contact with an object.

C2. Sensory system:
The sensory system measures the grip force exerted by the user's hand in order for the robot hand to grasp objects with different forces.

C3. Localization system:
The localization system identifies the spatial position and orientation of the user's hand relative to its original position.

C4. Power system:
The power system supplies electrical energy to the wearable device to power the system.

C5. Microprocessor:
The microprocessor reads sensor data and communicates with the communication system and sends control data to servo.

5. Software Architecture Diagram

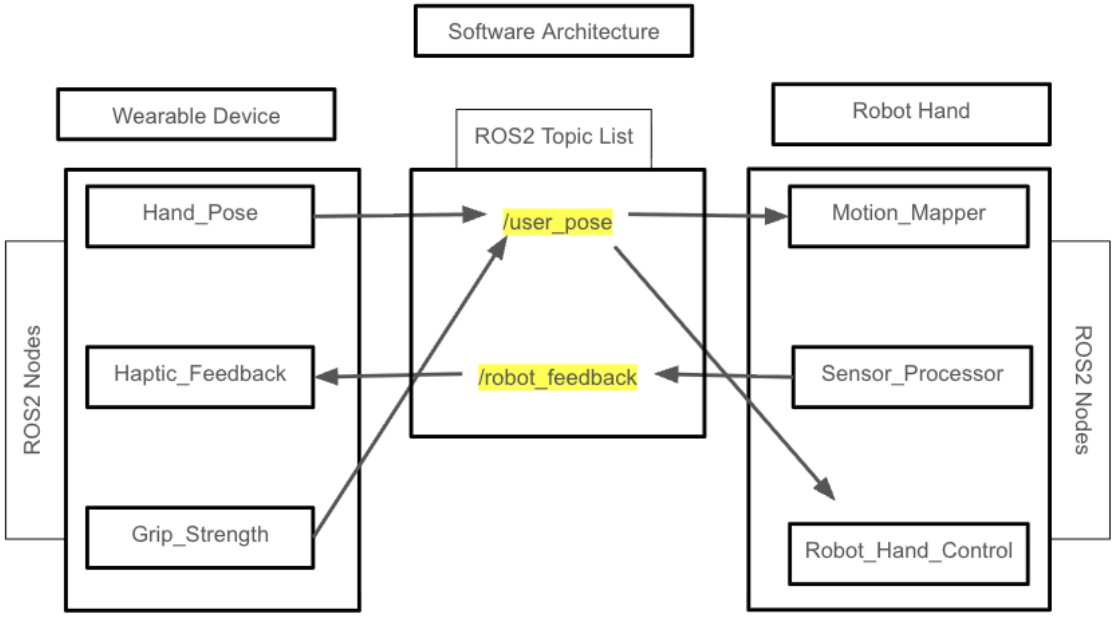
The software architecture for the system is organized into two primary subsystems: the Wearable Device and the Robot Hand. Each subsystem includes corresponding ROS2 nodes that communicate through shared topics to enable real-time teleoperation and haptic feedback.

On the wearable device side, the **Hand_Pose Node** captures the user's hand and finger motion and publishes this information to the **/user_pose** topic. The **Grip_Strength Node** measures the user's applied grip pressure and includes this data in the **/user_pose** stream as well, allowing the robot to mirror both motion and applied force. The **Haptic_Feedback Node** subscribes to the **/robot_feedback** topic to receive tactile and force information from the robot hand, converting these signals into corresponding physical sensations on the wearable device.

On the robot hand side, the **Motion_Mapper Node** subscribes to the **/user_pose** topic and translates the incoming user motion into robotic joint trajectories. The **Robot_Hand_Control Node** also subscribes to **/user_pose**, executing the commanded joint movements and regulating

grip strength. The **Sensor_Processor Node** continuously monitors the robot hand's contact and force sensors, publishing the recorded values to the **/robot_feedback** topic, which the wearable subsystem uses to generate haptic feedback.

Together, these nodes form a closed-loop communication system that allows the robot to accurately replicate the user's gestures while providing realistic tactile sensations to the operator. This bidirectional feedback loop maintains sub-200 ms latency, enabling stable and natural teleoperation.



6. Hardware/ Software Deployment

The system is divided into three main hardware components: the communication computer, the wearable device, and the robotic hand and arm. Each component runs a distinct layer of the control software, ensuring modularity, safety, and real-time responsiveness.

6.1. Communication Computer

Role: Central processing unit running ROS 2 and all high-level control software.

Functions:

- Receives sensor data from the wearable device and converts it into hand motion and position information.
- Translates wearable motion/position data into motor instructions for the robotic hand and arm.
- Sends commands containing target positions, control mode, and timestamps to the robotic subsystems.
- Switches between control modes:

    - Real-time motion following mode, where the robot mirrors the user's movements.
    - Position adjustment mode, where the robot gradually moves to a target pose.
- Receives feedback data from the robotic hand and arm (e.g., motor currents, fingertip pressure) and converts it into intended feedback signals.
- Sends corresponding feedback commands to the wearable device's haptic actuators.
- Manages all communication links and logs operational data for analysis and debugging.
- Provides real-time visualization through diagrams and 3D models for both user feedback and developer diagnostics.

6.2. Microcontroller on Wearable Device

Role: Sensor acquisition and haptic control.

Functions:

- Collects sensor data (flex, IMU, and force sensors) and sends processed motion data to the communication computer.
  Receives feedback commands from the computer and drives haptic actuators with controlled force changes to ensure safety and comfort.
- Reports command execution status back to the computer.
- Includes a safety monitor: when communication is delayed or lost, the system automatically releases all feedback forces to prevent unintended actuation.

6.3. Microcontroller on Robotic Hand and Arm

Role: Low-level actuator control and safety enforcement.

Functions:

- Collects feedback sensor data (joint motor encoders, fingertip pressure sensors) and transmits it to the communication computer.
- Interprets incoming commands and executes them appropriately in both real-time and position modes.
- Performs local PID control to move arm motors to target positions or follow trajectories smoothly and safely.
- Controls hand motors to mimic the user's hand motion accurately.
- Reports command execution status and system health to the computer.
- Contains onboard safety logic:
    - Detects communication delays or losses and gradually moves the robotic hand and arm to a safe "home" position.
    - Rejects unsafe commands (e.g., motions exceeding speed or joint limits) and reports risks to the computer.

7. Design Relative to Requirements

7.1. Safety and Fault Handling:

This is our highest priority and thus we will make our design decisions based around this.

a. We will require that our robot will make safety checks before it moves. This will drive the design of the processor, it will take data from the controller, make sure it is safe and will not collide with the environment in unplanned ways and only then issue instructions.
a. In the case of disconnection, there will be a fallback behavior for both the wearable device and the arm/hand. This will require checks before executing any commands and making sure that the feedback cannot be harmful to the operator.
b. The system will also reject connection requests from unauthorized users.
c. If anything goes wrong there will be an emergency stop that will return the controller and arm/hand to safe behavior.

7.2. Operator Intent Capture:

a. The system will capture the operator's hand, finger, and arm motion using a wearable device. We will design a wearable device that through various sensors to measure position, force, acceleration, ect will capture the intended actions of the user.
b. The system shall support calibration and re-centering between wearable devices and the arm/hand. We will include a way that the user can adjust the zero/default positions and orientations of the hand and each of the finger

7.3. Motion Mapping and Control:

a. The roboting arm can mirror or be commanded to move to the proposed end effector position by the user. We will make sure that latency of the system is sufficiently low to allow for real time usage and adjustment by the user.
b. We create a way for the user to toggle between real-time tracking and a more gradual go to and mimic gestures mode.
c. Before commands are given to the arm/hand, inputs will be received, converted to commands that should be given to the arm/hand, validated that the intended motions are safe, and then sent. This is to make sure the robot does not collide with the environment, exceed joint position or acceleration limits, etc.
d. We will design a hand that can detect force so that it can securely grasp objects of different sizes, weights, and texture.

7.4. Sensing and feedback:

a. We will include some force sensor in the fingertips so that it can convey that information to the user. The exact kind and method of conveying information to the user will be decided by prototype testing
b. Information about the position, force, and surface properties will be conveyed to the user in real time. The design will be low latency so that real time feedback can be applied and will include adequate sensors.

8. Concept Trade Studies

8.1. Location of Computation

8.1.1. Problem Description

A key design decision concerns where to perform the main computation and control logic: on distributed microprocessors located on each device, or centrally on the communication computer. This choice affects latency, synchronization, power use, system complexity, and ease of development.

8.1.2. Option 1: Computation on Microprocessors

In this design, each subsystem includes its own microprocessor: one on the wearable device and one on the robotic hand. The wearable microprocessor processes sensor data such as hand position, gesture, and force intent, then sends high-level control commands to the robotic hand. The robotic hand's microprocessor processes feedback from tactile and position sensors, interprets incoming motion commands, and directly controls its motors and actuators.

Advantages:

- Reduces communication load by handling local processing and control loops near the sensors and actuators.
- Enables faster local response and more resilient operation if communication temporarily drops.
- Improves modularity—each subsystem can operate semi-independently and be tested separately.

Disadvantages:

- Increases hardware and software complexity, as multiple processors must be programmed, synchronized, and maintained.
- Harder to update algorithms or debug across devices.
- Slight variations in timing between processors may introduce synchronization errors between the wearable and the robotic hand.

8.1.3. Option 2: Computation on Communication Computer

In this design, the communication computer performs all major sensor data processing, motion planning, and motor trajectory generation. The microprocessors on the wearable device and robotic hand only handle low-level interfacing with sensors and actuators, ensuring that motors follow the desired trajectories through local PID control.

Advantages:

- Simplifies system architecture and reduces the need for distributed synchronization.
- Easier to modify algorithms, visualize data, and log results from a single point.
- Provides higher computational capacity for real-time filtering, control, or vision tasks.
- Facilitates coordinated control and unified timing between the wearable and the robotic system.

Disadvantages:

- Increases dependence on continuous, low-latency communication links; performance may degrade if network conditions worsen.

- Centralized computation introduces a single point of failure. The other system can be set into a safe mode if connection or the communication computer fails, but they can no longer perform any tasks.
- May lead to higher bandwidth usage and slightly higher end-to-end latency if not optimized.

8.1.4 Design Decision

After evaluating both approaches, the team has chosen to perform the main computation on the communication computer. This allows more flexible software development, easier debugging, and better synchronization between subsystems, while keeping low-level stabilization loops on the microcontrollers to ensure smooth and safe actuator control. This centralized approach directly supports Communications & Interfaces (C2) (maintaining time synchronization across wearable, robot, and logs) and (C3) (logging operator intent and robot state). Furthermore, leveraging the processing power of the central computer satisfies Compatibility (X1) (operating on macOS/Apple Silicon) to handle complex kinematics within the latency limits of Performance (P1) (end-to-end control loop latency $\leq$ 200 ms).

8.2 Fingertip Pressure Sensor Choice

8.2.1. Problem Description

A key design decision concerns how to capture contact pressure on the fingertips of the robotic hand. This choice affects sensing accuracy, spatial resolution, mechanical durability, and suitability for closed-loop haptic feedback. The following options are evaluated: (A) Force-Sensing Resistors (FSRs), (B) Embedded Strain Gauge / Load-Cell Sensors, and (C) Fluidic / Pneumatic Channel Tactile Sensors.

8.2.2. Option A: Force-Sensing Resistors (FSRs)

In this design, each fingertip contains a thin resistive film that changes resistance when force is applied. The signal is read through a simple voltage divider and converted into analog force values. These sensors are widely used in low-cost robotic grippers and prosthetic devices for detecting grasping pressure (Lee & Nicholls, 1999).

Advantages:

- Low cost, compact form factor, and easy integration under soft fingertip pads.
- Requires minimal circuitry and processing; suitable for basic contact detection.
- Provides adequate normal-force estimation for simple grip feedback.

Disadvantages:

- Limited accuracy and non-linear response, with drift over time and temperature.
- Susceptible to wear under repetitive or sliding contact, reducing lifespan.

8.2.3. Option B: Capacitive Force Sensor

Capacitive force sensors consist of two parallel metal sheets separated by a dielectric. As force is applied to the sensors the dielectric deforms pushes the metal sheets closer together. As this happens the capacitance between the metal sheets changes predictably with force applied.

Advantages:

- Compact
- No moving parts
- Good dynamic response

Disadvantages:

- Measuring capacitance is not as straightforward as resistance
- Dielectric properties changes with temperature and over time
- More expensive than resistive

8.2.4. Option C: Fluidic / Pneumatic Channel Tactile Sensors

An emerging alternative uses soft elastomer fingertips embedded with fluidic or pneumatic channels. Contact pressure causes the internal pressure or flow distribution to change, which can be measured to infer contact location and magnitude (Zhou et al., 2025).

Advantages:

- Naturally compliant and durable under repeated contact or impact.
- Capable of distributed pressure sensing across the fingertip surface.
- Provides a more human-like tactile response that supports gentle manipulation.

Disadvantages:

- Less mature technology; requires custom fabrication and complex calibration.
- Slower response due to fluid compressibility and mechanical damping.
- May require learning-based methods for precise force estimation.

8.2.5 Design Decision

Considering integration difficulty, accuracy, and development schedule, the team will prototype using both Option A (FSRs) and Option B (Capacitive Sensors). The prototypes will be evaluated for signal quality, robustness, and ease of mechanical integration. Based on experimental results, one sensing approach will be selected for the final robotic hand design. This strategy ensures we meet Performance (P4) (pressure sensing resolution $\leq$ 1 N) and Sensing & Feedback (S2) (measuring fingertip grasping forces) while adhering to the Cost (C3) (robotic hand budget $\leq$ \$1,200). Option C was rejected to avoid complexity that could jeopardize Maintainability (M2) (sensors should be replaceable and modular).

8.3 Finger Actuation

8.3.1. Problem Description A key design decision is how to actuate the fingers of the hand. There are many mechanical systems that are able to produce linear or rotational motion. We will evaluate electric motors, pneumatics, and hydraulics for our application.

8.3.2. Option A: Electric Motors This is a common method that uses electric motors to either directly drive joints or to drive tendons attached to the joints. These are typically very precise and repeatable. Examples include the BarrettHand [7] and the Shadow Dextrous Hand [8].

- **Advantages:**
    - Precise
    - repeatable
- **Disadvantages:**
    - Gears can introduce backlash.

8.3.3. Option B: Pneumatics This method uses compressed air to actuate pistons or artificial muscles. This allows for naturally soft and compliant grasping, which can be advantageous. An example is the Festo Bionic hand [9].

- **Advantages:**
    - Lightweight and fast.
    - Naturally compliant because air is compressible.
- **Disadvantages:**
    - Less repeatable than motors because it's difficult to know the exact volume of air in the piston.
    - Requires tubes, valves, and compressors.

8.3.4. Option C: Hydraulics This method uses pressurized liquid and is more often used in high-force situations. An example is the Sarcos Guardian exoskeleton [10].

- **Advantages:**
    - More power-dense compared to other methods because pumps can be placed elsewhere.
    - Capable of very high force output.
- **Disadvantages:**
    - Bulky and very heavy.
    - Requires complex plumbing.

8.3.5. Design Decision

Based on our industry research, Option A (Electric Actuation) seems like the best option. Hydraulics (Option C) specialize in high-force applications, whereas we are more interested in fidelity. Though pneumatics (Option B) offer safety in the form of compliance, using electrically actuated fingers with force sensors should rid us of that problem. Also, the wires needed for electrically actuated systems are much smaller and more durable than hoses for pneumatic and hydraulic systems. Batteries/power supplies are smaller and quieter than pumps and valves. This

choice provides the precision required for Performance (P2) (end-effector pose tracking accuracy ≤ 10 mm) and Motion Mapping & Control (M5) (continuous control of grip gestures). Furthermore, avoiding fluid-based systems improves Maintainability (M1) (easy to clean and recover from damage) and ensures the hardware is robust enough for Reliability (R1) (withstand high loads without leaking).

8.4 Sensors for Capturing User Hand Position

8.4.1. Option A: Vision-Based Systems Vision-based hand pose sensors typically combine 2D (RGB/IR) and 3D (ToF/Stereo) sensors to estimate hand and finger position and gestures. Examples include the Apple Vision Pro [11] and the Microsoft Azure Kinect [12].

- **Advantages:**
    - Requires no gloves or sensors for the user to wear.
    - Can have a high update rate.
- **Disadvantages:**
    - Line of sight is required.
    - Lots of computation is required.
    - Presents difficult software challenges.

8.4.2. Option B: "Data Gloves" These consist of gloves that the user wears with sensors that estimate the position of the user's hand. They often use IMUs to measure acceleration, orientation, and estimate position. Finger bending can be estimated using flex sensors, resistive strain gauges, or fiber optic bend sensors. Examples include the CyberGlove III [13] (strain sensors) and the SenseGlove DK1 [14].

- **Advantages:**
    - Does not require line of sight to the hand.
- **Disadvantages:**
    - Hard to estimate absolute hand position and orientation.
    - Requires the user to wear a physical device.

8.4.3. Design Decision

based on our industry research, both gloves (Option B) and vision-based systems (Option A) have very attractive features for what we want. Vision-based systems are able to determine absolute position, whereas that is more challenging for a glove-like design. Glove-like designs are able to work without a direct line of sight, which is very important for us.

It would be advantageous to use a hybrid approach: use a vision-based system (Option A) for determining absolute position and orientation upon calibration, and use primarily IMU/flex sensor data from a glove (Option B) to find finger position, while still using vision for continuous position monitoring of just the hand. This hybrid approach ensures consistent compliance with Operator Intent Capture (I1) (capture arm/hand motions) and (I2) (detect

individual finger motions) even if one sensor type is obstructed. It also enhances Safety & Fault Handling (SF5) (safe fallback if tracking is lost) by providing redundant data streams to prevent total signal loss, which is critical for maintaining Usability & Availability (U1) (strong telepresence and low noticeable lag).

Trade Study	Options Evaluated	Key Trade-offs	Selected Design	Rationale & Requirements
8.1 Location of Computation	1. Microprocessors: Distributed on device/hand.2. Comm. Computer: Centralized processing.	Micro: Low comms load, modular; complex sync.Comm Comp: Simplified arch, powerful; comms dependent.	Communication Computer	Enables flexible dev and unified timing. Supports C2, C3 (sync/logs), X1 (macOS compat), and P1 (latency ≤200ms).
8.2 Fingertip Pressure Sensors	A. FSRs: Resistive film.B. Capacitive: Dielectric plates.C. Fluidic: Embedded channels.	FSR: Cheap, simple; non-linear.Cap: Robust, compact; complex readout.Fluidic: Compliant; slow, immature.	Prototype A & B (FSR & Capacitive)	Prototyping both to select best balance of integration/signal. Meets P4 (≤1N), S2, C3 (budget). Rejected C to save M2.
8.3 Finger Actuation	A. Electric Motors: Direct/tendon drive.B. Pneumatics: Compressed air.C. Hydraulics: Pressurized liquid.	Elec: Precise, clean; backlash.Pneu: Compliant; bulky, hard to control.Hyd: High force; heavy, leaks.	Option A (Electric Motors)	High fidelity and clean/compact power. Meets P2 (accuracy), M5 (control), M1 (maintenance), and R1 (reliability).
8.4 User Hand Tracking	A. Vision-Based: RGB/IR/ToF cameras.B. Data Gloves: IMUs & flex sensors.	Vision: Non-intrusive; LOS issues.Gloves: No LOS needed; drift, wearable required.	Hybrid Approach (Vision + Glove)	Combines Vision (absolute pos) and Gloves (finger flex/occlusion handling). Ensures I1, I2, SF5 (redundancy), and U1.

9. Prototyping

9.1 Finger Actuation Methods

Option A: Direct Drive

- **Setup:** Create a finger in which the motor directly (or through gears) actuates each joint.
- **Testing Criteria:**
    - Repeatability
    - Range of Motion
    - Max Torque (at different points)

Option B: "Tendon" Drive (Electric, Pneumatic, or Hydraulic)

- **Setup:** Create a finger that is actuated by strings/tendons attached to the finger segments.
- **Actuators:** Use motors or linear actuators to move the tendons.
- **Testing Criteria:**

    - Repeatability
    - Range of Motion
    - Max Torque (at different points)

9.2 Fingertip Pressure Sensing

- **Goal:** Determine the best small-package force sensor.
- **Test Setup:** Create a testing rig that can apply variable and repeatable force loads within the design specs.
- **Sensors to Test:**
    - Strain Gauges
    - Capacitive
    - Piezoelectric
    - Magnetic Force Sensors
- **Evaluation Criteria:**
    - Accuracy at **high** loads
    - Accuracy at **low** loads
    - Accuracy under **dynamic** (changing) loads

9.3 User Hand Position Sensing

- **Goal:** Evaluate sensors for capturing user hand position (noting Vision-based ToF is too expensive).
- **Methods & Setups:**
    - **Vision-based:**
        - **Setup:** Use a standard webcam video of the hand.
    - **IMU-based:**
        - **Setup:** Attach IMUs (Inertial Measurement Units) to the fingers and back of the hand.
    - **Bending-based:**
        - **Setup:** Attach different bending sensors to various locations on the hand.
- **Evaluation Criteria (for each method):**
    - **Absolute Position:** Move the sensors back and forth, ending in the same position, and measure the difference.
    - **Repeatability:** Move the hand around as it may be used, end in the original position, and measure the difference in sensor measurements.
    - **Accuracy:** Move the sensor a measured distance/angle and measure what the sensor reports.

Verification and Validation Plan

1. Functional Verification

The functional verification tests exercise each component and subsystem against the functional requirements to confirm they behave as intended. These tests will be conducted once individual components are available and integrated into their respective subsystems, ensuring every part is working correctly before full system integration.

1.1. Component tests:

- **Motors & Gearboxes:** Command known speeds/torques and measure output to confirm expected torque–speed behavior.
- **Servos:** Command target angles and verify they reach/hold positions within tolerance while resisting moderate external pushes.
- **Sensors (flex, IMU, encoders, force):** Compare readings against rulers, angle meters, and force gauges to confirm accuracy and repeatability.
- **Microcontrollers & Communication Links:** Run simple loopback and ping tests to confirm reliable message passing at the required update rate.

1.2. Operator Intent Capture Test

Covers: Operator Intent Capture (FR 1, 2, 3)
Goal: Verify that the system's processed representation of user intent (e.g., a pose/gesture/force-intent vector) correctly reflects what the user is trying to do, independent of low-level sensor noise.
Procedure:

1. Choose 4-6 canonical intents, e.g. relaxed, open hand, point index, grasp
2. For each intent, write a short, repeatable instruction for the operator (e.g., "open your hand fully").
3. For each intent, have the operator hold that posture / behavior for 2 seconds.

4. Take photos and measure the gesture of the operator as curvature and force
5. Store the user behavior of each internal representation as an intent data set that contains gesture (curvator) and force applied of each finger
6. Log the intent data set and compare it with the measured value
7. Repeat the test with at least 3 different people (to test robustness to hand shape and size)

Measurements & Success Criteria:

For all three users:

- The sensed curvature and force are both within 10% error rate from the real value
- For “light grip”, “medium grip”, “strong grip”, force sensed values must be strictly ordered and differ by at least 20% of full scale between levels.

1.3. Motion Mapping Test

Subsystem: Communication / control computer (mapping node)

Covers: mapping user intent to motor (FR 4, 6, 7)

Goal: Verify that internal intent representation is transformed into joint/servo command vectors correctly (e.g., correct finger mapping, scaling, and mode handling), independent of actual robot motion.

Procedure:

1. Use 8-10 intent samples spanning various finger gesture combinations and different grip-strength scalar values with the same format
2. The mapping node should map the 5 finger human hand gesture to the 4 finger robot hand gesture. Run the mapping node in simulation
3. For each input intent, record the output servo and motor command vector
4. For each sample, compute per-finger absolute error between expected robot finger angles and intent human finger gestures, and between expected robot finger force and finger force recorded

Measurements & Success Criteria:

- **Mapping error:** For gesture and force, the difference in expected value and intent should be within 10% of error
- **Constraint handling:** When intent implies unreachable posture (beyond joint limits), mapped commands must saturate at specified limits (never exceed allowed joint ranges).

1.4. Robotic Hand and Arm Control Test

Subsystem: Robotic hand & arm (actuators + microprocessor)

Covers: Motion Control & Grip Control (requirements M1,M2,M3,M4,M5,M6,C3)

Goal: Verify that the robot accurately follows joint/servo commands and can realize target finger forces and apertures.

Procedure:

1. Command the arm to 5 target end-effector position for the arm within its safe range
2. Command each finger to 5 target angles for each finger within its safe range

3. For each target, hold at the position for 3 seconds and record encoder data and actual angles
4. For arm joints, command slow sinusoidal trajectories (e.g., 0.2 Hz) from 20%–80% of range
5. Log commanded vs measured joint angles.
6. Mount a load cell under each fingertip.
7. Command target fingertip forces (1, 3, 5, 10 N) via the low-level grip controller. For each setpoint, hold contact for 5 seconds while recording force
8. Command continuous fingertip forces from 2 to 7 N within 10s, and log the force sensed at the load cell

Measurements & Success Criteria

- Steady-state error for finger angle $\leq 5°$ and end effector position error ≤ 2 inches L2 distance
- For trajectories, RMS tracking error $\leq 7°$ across the motion range.
- Achieved fingertip forces within ±10% of command at each tested level.
- The force exerted can change continuously with the difference in each discrete level less than 0.2 N

1.5. Fingertip Force Sensing Test

Subsystem: Fingertip sensors + local processing
Covers: Sensing & Feedback – force measurement (S1,S3)
Goal: Verify that fingertip force sensors report **accurate and repeatable values** across the expected operating range.
Procedure:

1. For each fingertip, press onto a calibrated force sensor or weights at 0, 0.5, 1, 2, 5, 10, 15, 20 N. At each level, hold for 3 seconds and record sensor output at 500 Hz.
2. Fit a line relating sensor output to applied force per fingertip.
3. For 3 selected levels (e.g., 1, 5, 10 N), apply and release force 10 times, recording peak readings.

Measurements & Success Criteria:

- $R^2 \geq 0.9$ for linear fit over the 0–20 N range.
- Mean error at each test level $\leq 10\%$ of applied force
- Standard deviation over 10 repeats $\leq 5\%$ of applied force at each test level

1.6. Haptic Feedback Mapping and Execution Test

Subsystem: Haptic feedback mapping node + wearable actuators
Covers: Sensing & Feedback – mapping & rendering (Requirements S1,S2,S3,P3,P4)
Goal: Verify that fingertip forces at the robot are transformed into proportional, timely, and distinguishable haptic feedback on the user's hand.
Procedure:

1. Command robot to apply forces of 0.5, 1, 3, 5, 10 N at a fingertip against a calibrated pad, log robot force sensor output and wearable actuator command
2. Attach a small force/acceleration sensor to the wearable at the feedback location, or use a test rig that measures actuator output (e.g., pressing on a secondary load cell). For each force level, log actuator output.
3. Ask a human subject to rank perceived strength of feedback for 0.5, 1, 3, 5, 10 N trials (presented in random order).

Measurements & Success Criteria

- Wearable output must be strictly increasing with robot fingertip force (no inversions)
- For each level, measured actuator output within ±15% of nominal scaling law
- Human ranking is correct in ≥ 80% of trials ($0.5 < 1 < 3 < 5 < 10$ N)

2. Performance Testings

After the testing on the communication, control, motion mapping, haptic feedback, etc. subsystems, we would like to examine the overall system performance on the two major operating modes— real time following, and human handmotion playback. Specifically, the system will be tested upon the success rate of performance and the communication latency.

Mode 1: Real time following

Testing procedures:

- Ask the user to perform a designated hand motion that incorporates several gestures, each targeting different d.o.f. of the robotic hands(raising one finger, raising two fingers, opening and closing the hand, etc).
- Record the motion of the robot hand and the human operator's hand through the camera. Analyze whether they are in sync with each other, and what is the time drift/latency between the two.
- Determine whether the trial is successful. We define the success trial as being able to perform the entire motion correctly and under a latency rate less than 300 ms.

The testing procedures will be performed at least 10 times with different operators to test the success rate. We will also ask the user to perform the motion in slow, medium, and fast rate(measured by the keyframe's time difference) to measure the relationship between the speed of the human motion and the latency and success rate of the robot system. We expect the robot will have a success rate of 90% under slow, and medium motion. (requirement p1)

The human motion can also be designed in different ways to capture the robot success rate towards different motions. For example, designing the motion like "crashing one grape and then holding the other grape" will allow us to test whether the robot is able to perform the user's intent correctly(requirement P2).

Mode 2: Human hand motion playback

The testing will be similar to mode 1. We determine the success trial as the robot is able to perform the entire action with a similar motion rate(with a ± 2 second allowance).

3. Failure Modes

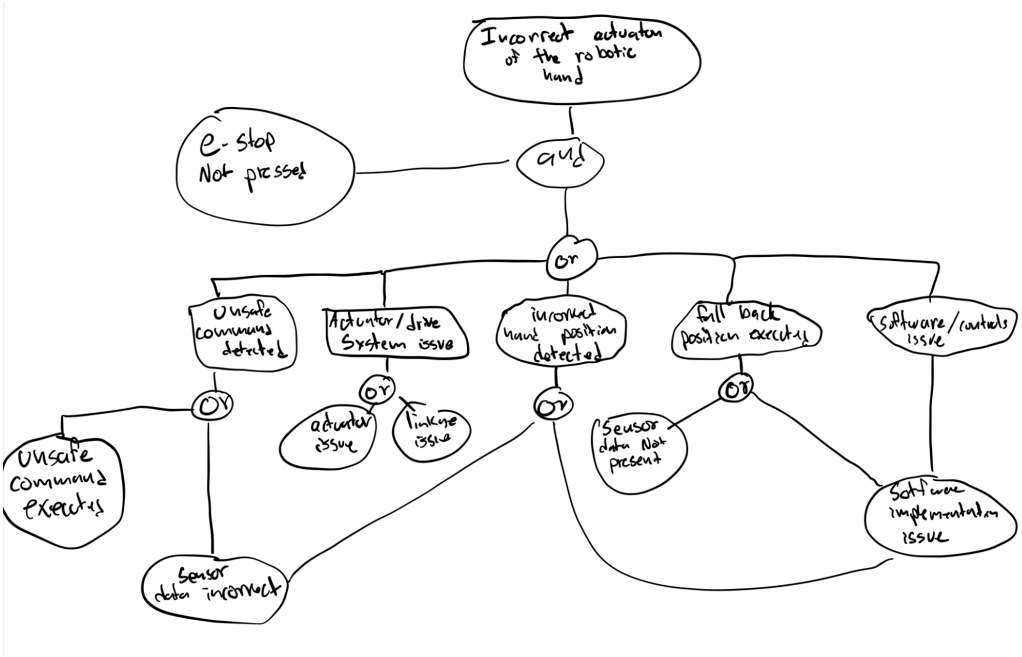
Failure Mode	Effect	Criticality
Incorrect or no detection of hand position	Data that is inconsistent with the true position of the users hand will be sent to the computer	Incorrect position data will cause the robotic hand to move to a not desired position. This can potentially be dangerous to things in the environment of the robotic hand
Incorrect actuation of the robotic hand	The Robotic hand will be in a position that is undesired	The undesired position of the robotic hand could be harmful to the environment.
Robotic hand actuators broken	One or more of the fingers will end up in an unexpected position	The robotic hand could fail to perform its task, not move at all, or drop something.
Incorrect force sensor data	A force that is inconsistent with being felt by the hand will be sent to the computer.	This may cause the user to believe they have grasped something they haven't or haven't grasped something they have. This likely will lead to an operation not being successful.
Incorrect feedback actuation	The feedback actuators will send information inconsistent with what is happening to the user.	This may cause the user to believe they have grasped something they haven't or haven't grasped something they have. This likely will lead to an operation not being successful. If the actuation is too strong this may hurt the user.

Safety Hazard	Description	Method of Mitigation
Unexpected motion	If there is a communication delay, loss, inaccuracy, or software malfunction the robot can move unexpectedly. It can collide with its environment, itself, or other things. For the user, sudden haptic feedback could be harmful.	Implement software limits for joint angles, velocities, and forces. Require an emergency stop that is accessible to the operator. Incorporate fallback actions in case sensor data is unreasonable or not present.
Excessive grip force	If the force sensing breaks or fails, the robotic hand may apply excessive grip force when manipulating objects. This can be harmful for the user, environment, and the robot itself.	Test force sensors and calibrate force sensors to make sure that data is accurate. Make the fingers compliant so to make the robot “soft” Include current/torque monitoring for the motors as

		well as direct force sensing
Electrical hazards	If haptics are damaged, there is a risk of electric shock to the user. Electronics on the controller may also overheat causing damage to the user.	Ensure proper electrical insulation and grounding of all the wearable components. Include current limiting circuits Stress test all electronics on the controller for safety

Fault Tree for Incorrect position of the robotic hand:

Incorrect position of the robotic hand could happen for a number of reasons. These can only happen if The e-stop is not pressed because if the stop is pressed the the hand will stop whatever its doing and wherever it does top is its intended position. If an unsafe command is detected the hand will not do the intended wrong command. An unsafe command can happen if an unsafe command is executed by the controller or if the sensor data incorrect. An incorrect position could be reached if there is a mechanical issue in the hand. This can either be an issue with the actuator or the linkages in the hand itself. If the controller detects an incorrect hand position the robotic hand could also end up in the wrong position.
This would either be due to incorrect sensor data or an issue with processing the sensor data in the software. Another way that the incorrect hand position could be executed is if the fallback position command is executed. This will happen in there is no sensor data or there is a software issue that causes it to think that there is no sensor data. Lastly if there is a software issue relating the sensor data to forward kinematics for the hand, the fingers or hand itself could end up in the wrong position.



Test	Test Procedure	How it Catches the Failure
Static & Dynamic Accuracy Test	Mount the wearable controller in a jig with known angles and compare sensor data to the jig's ground truth.	This test catches an incorrect position detection failure by comparing the wearable's sensor output to a known, true measurement. A large error indicates the sensor data is failing to reflect reality.
Unreasonable Data Test	Write a test script that feeds junk data that is impossible directly into the motion planner node.	This test validates the control mitigation. It confirms that nonsensical position data forces the robot into a safe, paused mode, catching the system's failure to handle the bad data before it causes movement.
Joint Limit & E-Stop Tests	Send a command past the software limit. Physically press the E-Stop button during motion.	This test validates the safety controls for unexpected motion. It ensures that commanding an unsafe move is rejected by the software. It also confirms the E-Stop successfully overrides all software to halt any dangerous, unexpected motion.
Communication Loss Tests	While the system is operating, physically unplug the USB or network cable connecting to the robot, and then separately, the cable to the wearable.	This test validates the "fallback" mitigations. It confirms that a total communication loss is properly detected by the onboard microcontrollers, which must then trigger their safe "home" position.

Force Calibration Test	Press a calibrated load cell or known lab weights against the robot's fingertip and record the sensor's reading.	This test catches incorrect force data by applying a known, measured force and checking if the robot's sensor reading matches. A significant mismatch indicates the sensor is failing.
User Safety Test	Have a user wear the device and command the maximum possible feedback signal. Ask the user to report if the sensation is painful or harmful.	This test validates the safety of the haptics. By commanding the maximum possible haptic output, it confirms with a user that the sensation is not painful or harmful, catching a failure where the actuation is dangerously strong.
Thermal Stress Test	Run the system in a continuous, high-load loop for a long time. Monitor component temperatures with a thermal camera or thermocouple	This test catches the overheating hazard by running the system at full load. A temp sensor is used to detect if any component exceeds its safe temperature limit, which would indicate a fire or damage risk.
Redundant Sensor Check	While grasping an object, log both the motor current data and the fingertip force sensor data. Plot them against each other in real-time or after the test.	This test catches a failing actuator during operation by comparing two different sensors. If the motor current is high but the force sensor reading is zero , this discrepancy indicates a mechanical break.

4. Requirements Validation

Requirements Validation confirms that the system meets its listed functional and non functional requirements and that each requirement has a quantitative validation metric and a defined measurement procedure. The traceability matrix below outlines these metrics, describes the methodology used to test them, and tracks the validation status of each requirement.

Req ID	Requirement Description	1.1 Components	1.2 Intent	1.3 Motion Map	1.4 Robot Ctrl	1.5 Force Sense	1.6 Haptics	2.0 Performance	3.0 Failure
I1	Measure operator arm/hand motions		**X**						
I2	Detect finger motions & intended forces		**X**						
I3	Support calibration & re-centering		**X**						
M1	Mirror operator motions in real-time				**X**			**X**	
M2	Move to end-effector position				**X**			**X**	
M3	Support two control				**X**			**X**	

	modes								
M4	Block unsafe/infeasible commands			**X**	**X**				**X**
M5	Continuous control of grip & force	**X**			**X**				
M6	Securely hold objects (Payload)	**X**			**X**			**X**	
S1	Detect contact at fingertips					**X**	**X**		
S2	Measure fingertip grasping forces					**X**	**X**		**X**
S3	Provide real-time feedback					**X**	**X**		
C1	Bidirectional communication							**X**	**X**
C2	Maintain time synchronization							**X**	
C3	Log internal knowledge/topics				**X**				
C4	Visualize real-time pose/sensor data							**X**	
SF1	Safety checks before enabling motion								**X**
SF2	Reject unauthorized connections								
SF3	Prevent unsafe collisions								**X**
SF4	Safe fallback for Robot (Comm loss)								**X**
SF5	Safe fallback for Wearable (Comm loss)								**X**
SF6	Emergency stop accessible								**X**
P1	End-to-end latency ≤ 200 ms							**X**	
P2	End-effector tracking accuracy				**X**			**X**	
P3	Grip force range resolution	**X**					**X**		

P4	Pressure sensing resolution	**X**					**X**		
P5	Robot hand payload capacity				**X**				
P6	Arm workspace radius				**X**				
P7	Haptic feedback update rate						**X**		
U1	Telepresence (User ranking)						**X**		
U2	Availability (System uptime)								**X**
R1	Reliability (Thermal/Stress)								**X**
R2	System lifetime target								**X**

Detailed Design

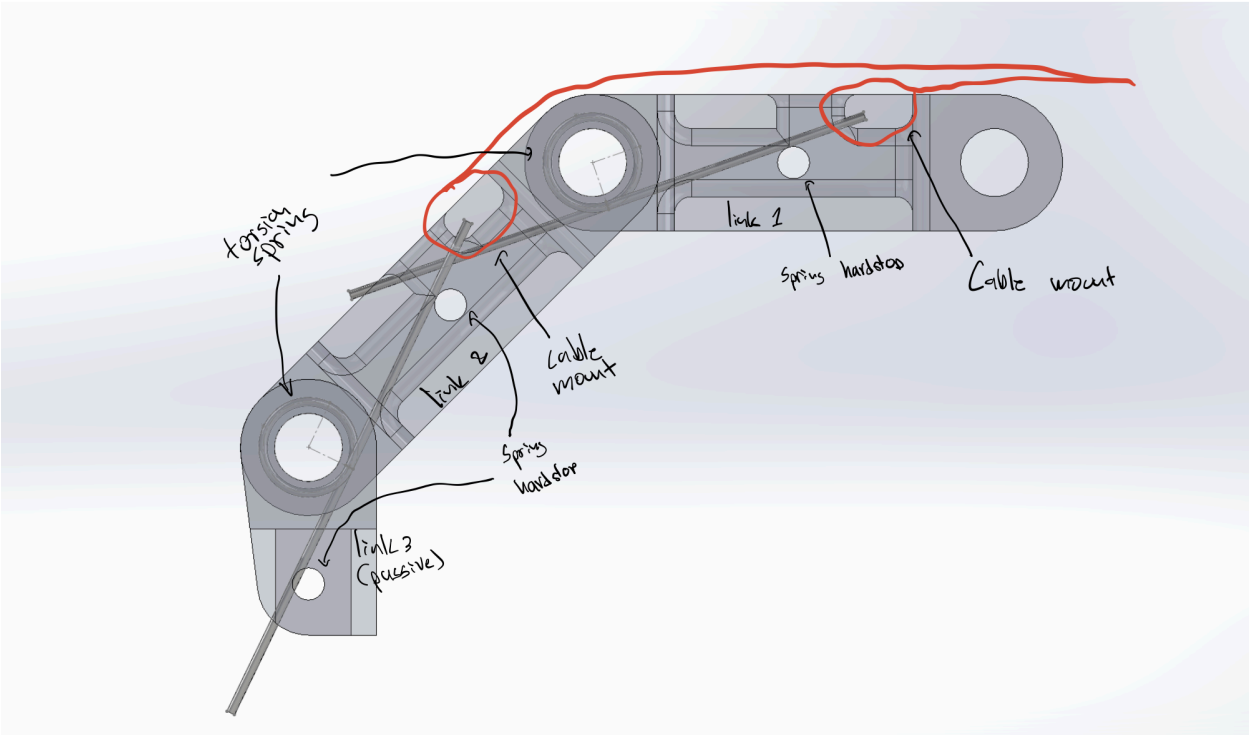
1. System Description:

The Teleoperated Hand System enables a human operator to manipulate objects on a tabletop using a robotic arm and hand that mirror the operator's motions in real time. The system is used in a lab environment where the robot arm, robotic hand, and host computer are mounted on a workstation, and the operator stands nearby wearing a cable-driven haptic glove. A Leap Motion Controller provides 3D tracking of the operator's hand.

To make teleoperation intuitive, the system does relative motion mapping rather than full-arm imitation. After an initial calibration, the robot arm moves so that the change in the operator's hand position and orientation corresponds to an equivalent change in the robot hand's pose. This means the robot does not need to replicate the exact geometry or posture of the human arm, only the operator's intended motion direction and magnitude.

Finger motions are captured and translated into tendon-driven finger actions, allowing the robot hand to perform grasps, pinches, and other fine manipulations. When the robot contacts objects or reaches workspace limits, the glove provides force feedback so the operator can feel resistance. Together, these components create a natural and safe teleoperation experience suitable for tabletop manipulation tasks.

2. Scaled Drawings of the System and Components:

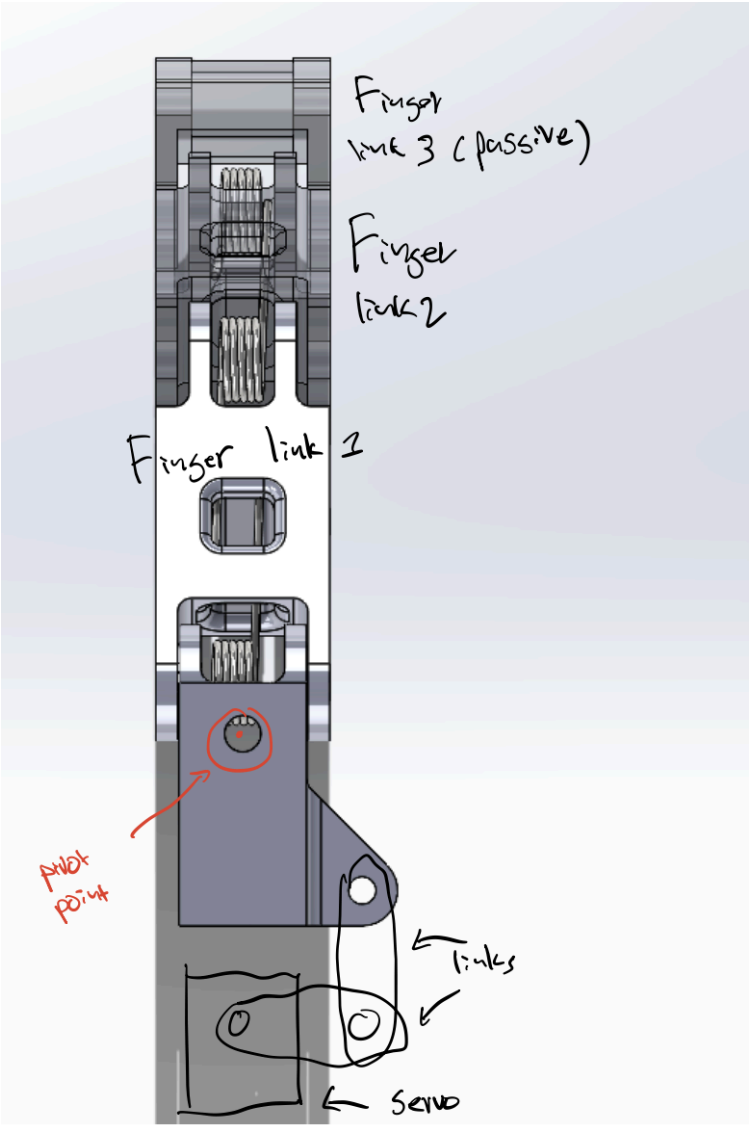


Finger CAD

This is an image of one of the fingers; it consists of three links similar to a human finger. Each link is contracted using a torsion spring and retracted using a cable/tendon system. The red lines represent the cable routing. The last link is passive—no tendon mount, only hardstops. The hand closes passively using the torsion springs and opens using the tendons. This simplifies the design by removing the need for dual-sided tendons or direct-drive joints while still enabling the system to mirror user hand motions in real time .

The links rotate about quarter-inch shafts that the torsion springs rest on. The links are hollow so that the spring legs sit inside the 3D-printed geometry. There are 3mm holes for shoulder screws that act as the contact points for the torsion-spring legs. The torque produced by the spring allows each link to naturally contract, supporting detection of intent forces and finger-motion cues through cable tension as the system responds to user gestures.

All fingers mount to the hand using this side-to-side mechanism, allowing fully independent lateral motion for each finger. This design directly supports Motion Mapping & Control (M5), which mandates that the robotic hand allow continuous control of grip gestures and aperture. Cables are routed over the base and servo assembly and actuated further back using a pulley system. This actuation strategy is critical for Reliability (R1), ensuring the hardware withstands loads, while the modular nature of the routing supports Maintainability (M2), allowing fingers to be replaceable. Reliable routing and actuation help ensure that if the sensing system ever becomes unreliable, the system can execute a Safety & Fault Handling (SF4) protocol, entering a safe fallback state (holding or returning home) without mechanical failure.



Top View of Finger CAD

This image illustrates the mechanism for side-to-side finger rotation. The finger assembly attaches to a base with a vertical pivot point. Unlike the flexion joints, the lateral rotation is directly driven by a servo instead of tendons. Tendons were not used here because they can only pull in one direction; in contrast, direct drive allows full bidirectional movement and improves the system's ability to mirror operator intent precisely . Packaging constraints are less severe at the base, so using a servo here avoids unnecessary complexity.

All fingers mount to the hand using this side-to-side mechanism, allowing fully independent lateral motion for each finger. Cables are routed over the base and servo assembly and actuated

further back using a pulley system. Reliable routing and actuation help ensure that if the sensing system ever becomes unreliable.

3. Trade Studies

Camera trade study:
2D camera: Logitech C920 webcam [19]
3D stereo camera: Leap motion controller [17]
RGBD camera: Microsoft Azure Kinect DK [12]

Sensor	Cost	Cost Score (6)	Accuracy	Accuracy Score (10)	Compute	Compute Score (8)	Total Score
Logitech C920 webcam (2D camera)	$**55**	10	**Inaccurate** because 2D cameras estimate depth from algorithms	4	**Low compute** optimized models exist that can track motion in real time	10	**180**
Leap Motion Controller (2x2D IR stereo camera)	$**100**	8	**High accuracy** ,two high definition cameras. Depth estimation is great because of stereo	9	**Low compute** required, the device does internal IR imaging.	8	**202**
Microsoft Azure Kinect DK (RGBD camera)	$**400**	3	**Good Accuracy** - 2D resolution is good but Depth resolution isn't as good	8	**High compute** required, no onboard compute, GPU required for real time	0	**98**

This is a trade study of cameras that can be used to track hand positions. A 2D camera is a conventional camera that can tell the computer what color light is coming from what angle in front of it. It uses an image sensor that has millions of different small arrays that detect incoming

light to form an image. A 2D camera cannot tell how far the light came from , only its angle and wavelength.

A model processes this data and estimates the position and state of the hand, which directly supports the system requirement to capture the operator's hand and arm motions and detect finger motions. However, because depth is not measured, these estimates can sometimes be inaccurate or unstable, which impacts the ability to mirror motions reliably in real time and can contribute to motion ambiguity that affects safety feasibility checking . The primary benefits of 2D cameras are low cost, wide availability, and highly optimized models, which reduce processing demand and help maintain responsiveness. Because the models are trained on data of bare hands, a hand that has our haptic feedback system on it may not perform well.

Stereo cameras, such as the Leap Motion Controller, use two 2D cameras with a known baseline distance and angle between them. By comparing the differences between the two images, the system calculates object depth and therefore has more accurate knowledge of hand position. This improves the fidelity of operator motion capture and significantly enhances finger articulation detection . The additional depth information allows the robot to mirror user motions more accurately and smoothly in real time and gives the system clearer 3D geometry for evaluating safe or unsafe intents .

RGB-D cameras also provide color and depth but obtain distance differently. Systems like the Azure Kinect DK [12] use a 2D camera combined with a Time-of-Flight (ToF) infrared system that measures the return time of emitted light. This enables highly accurate depth measurements, again benefiting operator intent capture and finger-motion detection . However, the depth system cannot operate at full resolution at video framerates, which can lead to reduced depth fidelity or lag. These issues can hinder real-time mirroring responsiveness and increase the likelihood of tracking degradation, which must activate a safe fallback . The large amount of depth data also increases computational load.

We decided that the most important evaluation metrics were cost, accuracy, and computation. Cost matters because hand tracking is only one part of the system, and adhering to Cost (C4) (keeping the wearable device budget $\leq$ \$500) is critical. Accuracy is essential to have smooth and working motions to satisfy Performance (P2) (end-effector pose tracking accuracy $\leq$ 10 mm), and computation limitations affect whether the system can maintain stable tracking without triggering safety fallback. Based on the strengths of each option, the Leap Motion Controller was selected. It provides highly accurate depth-aware hand and finger tracking that best satisfies Operator Intent Capture (I1) (capturing operator arm/hand motions) and (I2) (detecting individual finger motions and intent forces). It supports Motion Mapping & Control (M1) (mirroring operator motions in real-time) with low computational demand, and behaves reliably enough to minimize the risk of tracking degradation triggering Safety & Fault Handling (SF5)

(safe fallback to release force if tracking is lost), all while remaining cost-effective. This makes it the optimal choice for this system.

Haptics trade study:
DOGlove [15]
Lucid Glove [16]
LRA(linear resonant Actuator) [20]

Haptic System	Cost	Cost score (6)	Feedback quality	Feedback quality Score (10)	Usability	Usability score(7)	Total score
DOGlove	~$**500**	3	**Excellent**, it applied a force in the desired direction on the desired finger through links	9	DOGlove is **Difficult** to use because the geometry of the link is incompatible with some hand gestures	3	**129**
Lucid Glove	~$**60**	9	**Great,** applies a force on the desired finger through cables	8	Lucid Glove is **Not Very Restrictive**, it is compatible with all hand gestures however friction from the cables would be noticeable	6	**176**
LRA	~$**100**	8	**Mediocre**, LRA are unable to apply a force on the finger, it varies the frequency and magnitude of vibration.	4	LRA is **Easy to Use**, LRAs are light weight and will never get in the way of any hand gesture, there is also no friction from feedback cables	9	**151**

We did a trade study on different ways to provide feedback to the user about what kind of force the robot hand is experiencing. We compared the DOGlove, Lucid Glove, and LRAs(Linear resonant actuators). The DOGlove and Lucid Glove are open source haptic force feedback systems, LRAs are motors that create a linear vibration, these can give the sensation of touching something by varying the frequency and amplitude of the vibration.

DOGlove is a system that attaches to the hand and has linkages that attach to each of the fingertips, each of the joints of the linkage has an encoder and the base joint has a servo. By controlling the angle of the servo the DOGlove can apply a force to the finger, and mimic the sensation of holding something in your hand. The DOGlove has encoders at each of the joints so it can also measure the position of each of the fingertips, this system would make our hand tracking a lot easier. However, from our prototyping tests we found that the linkages put hard limits on the finger positions, we found that some hand gestures such as a fully closed fits and fully open hand were not possible with the DOGlove.

Lucid Glove is also an open source haptic force feedback system like the DOGlove. In contrast to the DOGlove the Lucid Glove uses cable to provide the force feedback. The lucid glove uses sprung cable spools that can be locked and unlocked. The Lucid Glove is very flexible and will allow that hand to make any motion such as crossing one finger over the other and a fully closed fist–gestures that would not be possible for the DOGlove. However, this system has no way of measuring the exact finger position; the fact that the cables are spring would result in there always being some amount of force applied to the fingers.

LRA motors vary the frequency and amplitude of linear vibration in them. If we were to use this system we would have one in each of the fingertips which would directly communicate the force that the force-resistive sensor is measuring. This system would prove the most freedom with there being no resistance moving around and allowing the user to have any hand gesture.

We decided that the most important evaluation metrics were cost, feedback quality, and usability. Cost is always important for us because we have a few major subsystems, and it would be harmful to the project if one of them were to take up the lion's share of the Cost (C1) (total system cost $\leq$ \$2,500). The feedback quality is the most important to us because the goal of this project is to satisfy Usability & Availability (U1), which requires strong telepresence where the operator perceives realistic distance, pressure, and texture with low lag. Lastly, usability is also important because it is beneficial for the set of achievable hand gestures to overlap with the motions that the system can capture and reproduce, ensuring compliance with Motion Mapping & Control (M5) (continuous control of grip gestures). However, usability is somewhat less critical because some unreachable gestures, such as crossed fingers, are unlikely to be useful.

4. Parts List:

System	Part	quantity	notes
robot hand	Proximal link	4	3D printed. The first and longest link of the finger(active)
	middle link	4	3D printed. the middle link in the finger (active)
	distal link	4	3D printed. The third link in the finger (passive)
	thumb link	2	3D printed. Both thumb links (active)
	base rotation part	5	3D printed. Roates all of the fingers left to right
	torsion spring	15	is responsible for the clasping/closing of the hand motion for the tendon driven components
	cable	1	Will actuate the opening motion of the hand for the active joints. connects between the joints and servo
	servo	15	Servos are resposible for all of moving each of the active joints. Each finger is 3DoF including the thumb and there are 5 fingers.
	Palm/base structure	1	This will mount all of servos, and finger structures and will also be 3D printed.
	Teensy 4.1	1	microcontroller board for robotic hand
	Teensy shield	1	custom/prefab PCB for voltage divider (for Force-Sensitive Resistor), protection circuit and wiring around Teensy
	USB-micro to USB-A cable	1	Teensy to USB cable
	Force-Sensitive Resistor	5	For pressure detection of robot hand for detecting pressure on fingertips
	Resistors	5	For voltage dividor circuit for the FSRs
	buck converter DC-DC	1	Converts 22V to 5V for Teensy and Dr.Bot. Maybe LM2596S
	digital power monitor	5	To understand if the servos are blocked, maybe INA219
	capacitors	4	To protect the circuits, one for each voltage level
robot arm	power supply 22V	1	AC → 22 V DC supply
	BLDC joint actuators	5	Dr. Bot arm motors
	moteus-r4 motor controllers	5	Dr. Bot arm motor controllers
	CAN-FD to USB adapter	1	Dr. Bot arm communication signal line
glove(lucid glove 3)	ESP-WROOM-32 Dev Board	1	central control board
	WL b10k ohm Potentiometer	5	for gesture sensing

System	Part	quantity	notes
	retractable badge reels	5	for gesture sensing
	Nylon Inspection Gloves	1	for mounting everything on it
	3D printed parts for mounting parts on glove	1	for mounting parts on glove
	MG90S servo motors	5	for haptic feedback
	rechargable batteries	2	for power supply
	Breadboard Power Supply Module	1	converts battery input of 9V to 5V
wiring	jumper wires	a lot	
	JST connectors	some	
	E-stop button	1	put on servo power line to stop all servos when pressed
	Resettable polyfuse	1	put on 22V for input protection
	TVS diode	1	put across 22V and GND for input protection

5. Fabrication and Implementation:

Hardware fabrication:
The hardware for the robot hand consists of 5 fingers, a side to side rotation mechanism for each of the fingers, a palm-like structure that all of these mount to, and motors that drive all of the fingers. All of this is mounted to a robot arm.

Parts that will be 3D printed include:

- All of the finger linkages
- Rotating base for each of the fingers side to side motion
- Four-bar linkage for finger side to side motion
- Pulleys for finger joint actuation
- “Palm” for mounting the fingers and pulleys

Hardware such as bolts, torsions springs, cables, sensors will all be purchased

Our robot arm is borrowed from another class and is already assembled. It comes with a base that can be mounted to a table, 5 motors, joints, and motor controllers.

Fabrication and Implementation of Software:
<u>Shared Software Development and Version Control:</u>
All software for the system will be stored in a shared GitHub repository.
Team Members can:

- Develop features on separate branches (per module or task)
- Commit and push changes after major updates
- Use pull requests for code review before merging into the main branch

Primary Host Platform

- Operating System: macOS on Apple Silicon (Apple M2 Pro)
- Primary Host Languages: Python and C

The host computer (MacBook) runs the main control software:

- LeapInterface (C using LeapC API)
- TeleopCore (Python)
- MoteusArmController (Python)
- HapticsController (Python)
- GloveInterface (host side, Python)
- System monitoring / logging (Python)

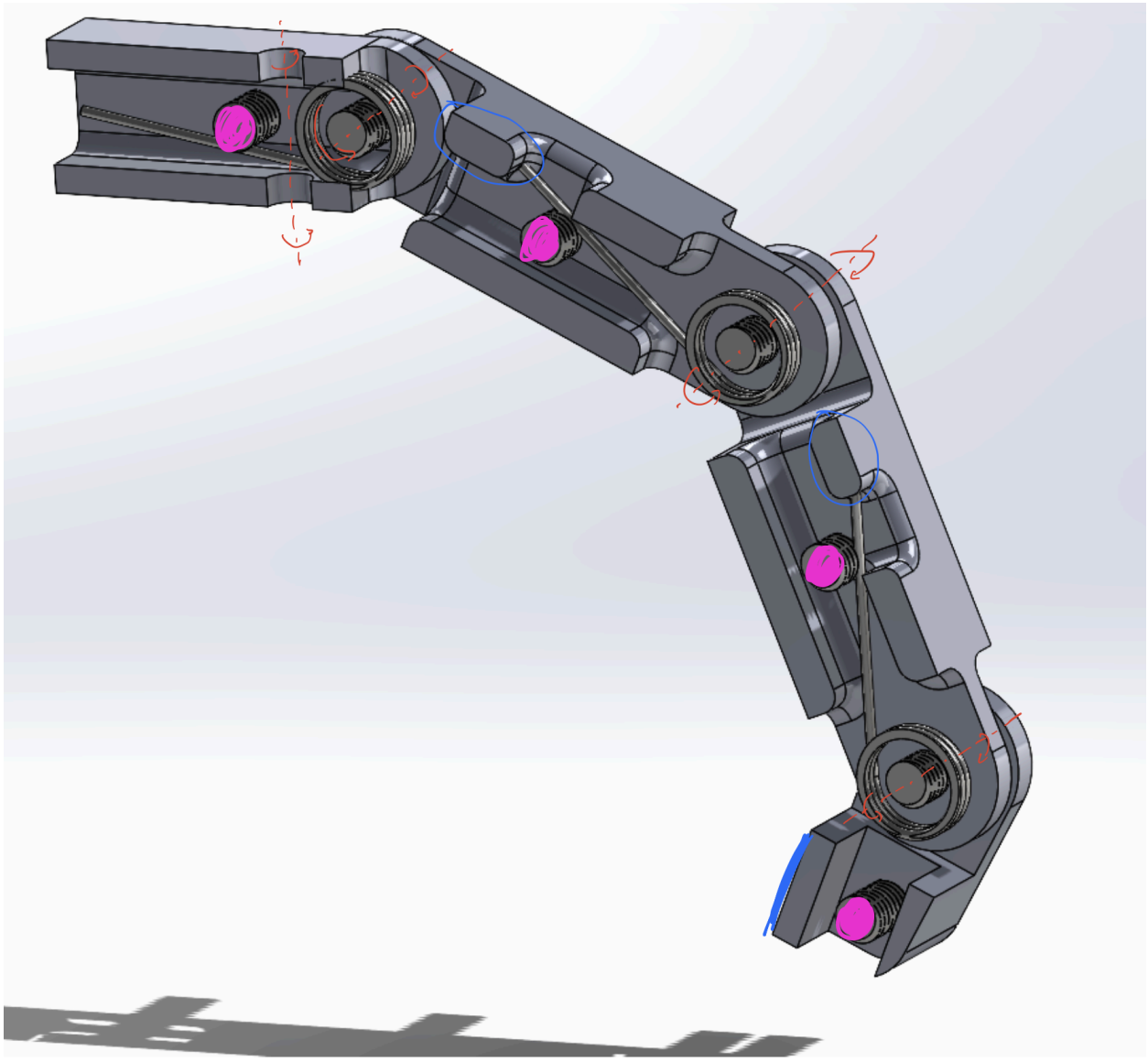
Embedded Platforms

- Microcontroller: Arduino Nano ESP32
- Firmware Language C/C++ using the Arduino IDE

The microcontrollers run firmware for:

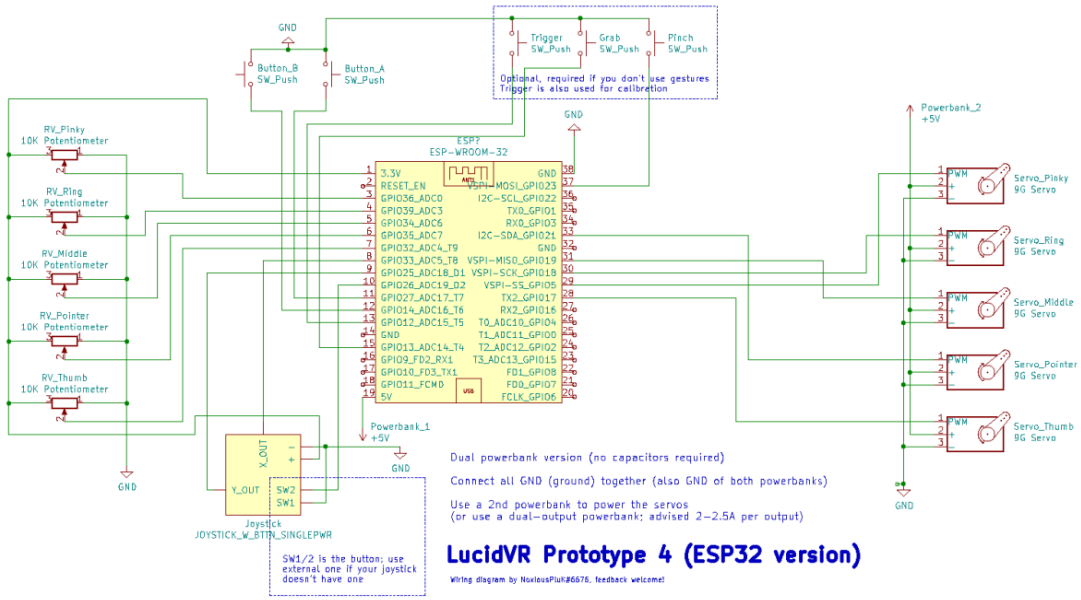
- HandServoController firmware (servo actuation for robot hand fingers)
- Glove firmware (cable-driven haptics for the glove)

6. Assembly instructions:

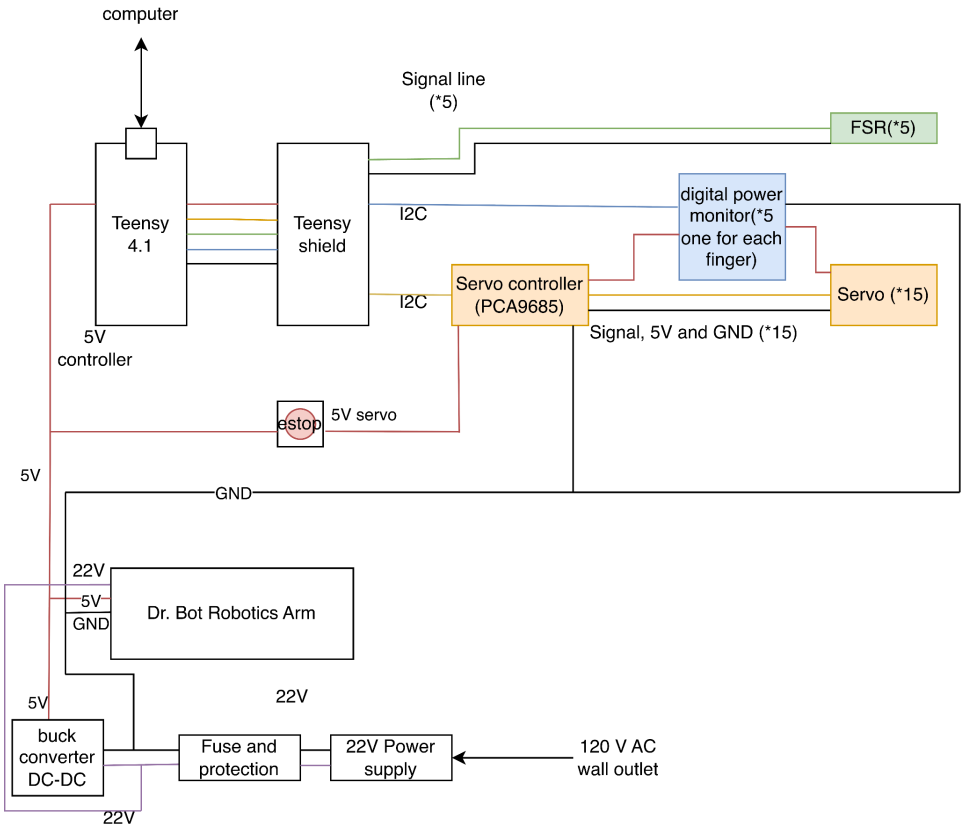


Cut View of Finger CAD Labeled

This is a cut view of the finger assembly in solidworks. To start assembling the first step would be to tie the cable around the sections marked with blue circles, these are the anchor points. The next step would be to put in the hard stop bolts for torsion springs that are marked with magenta. Next place the torsion springs in between the links at the joints making sure that the tangent part of the legs are facing down, this makes sure the torsion closes the hand rather than opening it. It is also important to make sure that the legs of the torsion spring are on below (inside) of the hardstop bolts, this will prevent the leg from digging into the print, it will also make sure that the torque applied by the spring is the desired amount. Next, put in the bolts that hold the torsion springs in place. Finally attach the base joint to the hand assembly so that the servo that moves that base is in a four-bar setup, and also attach the cables to their respective pulleys.



Wearable Device Wiring Diagram



Robot Arm and Hand Wiring Diagram

Electrical & Power Assembly Process (Arm + Hand)
The electrical system is built around the existing robot arm power rails. The arm's 22 V actuator bus and 5 V logic rail are treated as shared inputs to the robot hand electronics. The 22 V line (after the system E-stop) supplies all actuators, while the 5 V logic line powers low-power control electronics.

For the robot hand, a compact Teensy-based electronics board is mounted near the wrist or forearm. This board accepts two connections from the arm: 5 V logic rail, and ground. The existing 5 V logic line connects to the Teensy VIN pin, and the Teensy's internal regulator provides a 3.3 V rail for FSR dividers and any low-voltage ICs. A 5V rail is split for the servos. Basic protection components (inline fuse or polyfuse on the 22 V branch, a TVS diode, and bulk capacitors on the 5 V servo and 5 V/3.3 V rails) are placed on this board to handle inrush and transient events.

Servo and sensor interfaces are concentrated on the same board. Rows of 3-pin headers provide signal, 5 V servo power, and ground for each hand servo. Separate 2-pin headers provide connections for each fingertip FSR. The fixed resistors for the FSR voltage dividers are implemented on the PCB: each divider is tied to 3.3 V on the top, to ground through a resistor on the bottom, and to a Teensy analog input at the midpoint. The current sensors (INA219 modules) are inserted electrically between the 5 V servo rail and groups of servos, and connected back to the Teensy via the I²C lines exposed on the board.

Once the board is assembled, the hand wiring harness is built around it. Each servo in the palm region is connected with a short 3-wire cable terminated at the corresponding servo header. Each fingertip FSR is mounted under a compliant pad, with a light 2-wire cable soldered to the FSR tail and brought back to its sensor header. Cables are routed along the fingers, palm, and forearm using printed clips, zip ties, and sleeving so they flex with the joints and are protected from abrasion or pinch points.

At the system level, the 22 V actuator bus from the arm's power subsystem is routed through the main latching E-stop switch, then fanned out to the arm's BLDC motor controllers and to the hand's 22 to 5 V servo converter input. The 5 V logic rail is connected directly to the hand board's logic input so that the Teensy and sensors remain powered even when the actuator bus is interrupted. An additional contact on the E-stop is wired to a digital input on the Teensy as an E-stop sense line, allowing the controller to detect when actuator power has been removed and to hold all servo outputs in a safe state until the system is reset. After these connections are made,

the hand electronics board is secured to the arm structure with the USB port accessible for programming and future firmware updates.

7. Software Module Interaction Description and Diagrams:

Leap Motion Controller Module (LeapInterface) [17]

Purpose: The Leap Motion Controller Module is used to track the human operator's hand and fingers in real time.

The LeapInterface module connects to Ultraleap's Hand Tracking Service [17] using the LeapC C API, extracts the relevant hand pose information, and publishes a compact HumanHandPose representation that is used by the teleoperation and haptics modules.

External Dependencies:

- Hardware: Leap Motion Controller (Ultraleap)
- Software: Ultraleap Hyperion
    - Ultraleap Hand Tracking Service (running on macOS)
    - Leap SDK (LeapC C API), located at: /Applications/Ultraleap Hand Tracking Service.app/Contents/LeapSDK/lib
    - Ultraleap Control Panel (for device configuration and debugging)

Inputs and Outputs:

- Input: Tracking frames streamed from the Ultraleap Hand Tracking Service via LeapC
- Output:
    - HumanHandPose — a simplified hand pose structure consumed by the TeleopCore module to generate robot arm and robot hand commands

Use of LeapC / LEAP_HAND:

The LeapInterface uses LeapC API to get tracking frames and extract both spatial and grasp information from the LEAP_HAND structure.

Relevant fields from LEAP_HAND includes:

- Hand Identity:
    - hand->id - Unique identifier to track a hand across frames
    - hand->type - Distinguish left vs right hand
- Hand Position & Orientation:
    - hand->palm.position - XYZ position of palm center
    - hand->palm.orientation - Quaternion for palm rotation
    - hand->palm.normal - Normal vector of palm
    - hand->palm.direction - Forward direction of palm
- Finger Kinematics (per finger):
    - hand->{finger}.bones[0–3] - Bone data for each finger (thumb, index, middle, ring, pinky)

- Each bone (LEAP_BONE) contains:
    - position - Joint position (start of bone)
    - next_joint - End position of bone
    - direction - Direction vector along bone
    - width - Finger thickness
- Hand Pose/Gesture State:
    - hand->grab_strength - How closed the fist is (0-1)
    - hand->pinch_strength - Pinch gesture strength (0-1)
    - hand->pinch_distance - Thumb-to-index distance (mm)

HumanHandPose Representation:

- palm_position // 3D position of the palm (meters)
- palm_orientation // Orientation of the palm
- finger_flex[5] // Open (0) to closed (1) value for each finger
- grab_strength // Overall grasp strength (0–1)
- pinch_strength // Pinch strength between thumb and index (0–1)

Per-finger flex values are computed by normalizing joint angles derived from finger bone data between fully extended and fully flexed configurations.

Control Flow (High-Level):

At runtime, the LeapInterface module:

1. Opens a LeapC connection to the Ultraleap Hand Tracking Service.
2. Enters a loop where it polls for new tracking messages using LeapPollConnection().
3. For each LEAP_TRACKING_EVENT, it:
    a. Reads palm position and orientation
    b. Reads grab_strength and pinch_strength
    c. Computes per-finger flex values from finger bone data
    d. Creates a HumanHandPose instance from these values
4. Publishes the HumanHandPose to the TeleopCore at the designated tracking rate

Example Pseudocode for LeapInterface:

```c
// Pseudocode for LeapInterface using LeapC

LEAP_CONNECTION conn = OpenLeapConnection();

while (running) {
    LEAP_CONNECTION_MESSAGE msg;
    if (LeapPollConnection(conn, 1000, &msg) == eLeapRS_Success &&
        msg.type == eLeapEventType_Tracking) {

        const LEAP_TRACKING_EVENT* frame = msg.tracking_event;
        for (uint32_t i = 0; i < frame->nHands; ++i) {
            const LEAP_HAND* hand = &frame->pHands[i];

            HumanHandPose pose;
            pose.palm_position_m = mm_to_m(hand->palm.position);
            pose.palm_orientation = compute_orientation(hand);
            pose.grab_strength = hand->grab_strength;
            pose.pinch_strength = hand->pinch_strength;

            compute_finger_flex_values(hand, pose.finger_flex);

            publish_hand_pose(pose);  // to TeleopCore
        }
    }
}
```

TeleopCore Module (Teleoperation Mapping)

Purpose: The TeleopCore module is responsible for converting the human operator's hand motion into robot commands and switching between the various user modes.

It takes the HumanHandPose created by the LeapInterface and maps it to

- RobotArmCommand – how the 5-DOF Dr. Bot arm should move
- RobotHandCommand – how each robot finger joint and base yaw should move

TeleoopCore runs continuously on the host computer, reading the latest Leap data and streaming updated arm and hand commands at a fixed control rate.

External Dependencies:

- Software (internal modules)
    - LeapInterface – provides HumanHandPose
    - MoteusArmController – executes RobotArmCommand at the joint level
    - HandServoController – executes RobotHandCommand on the tendon-driven fingers
    - HapticsController – uses TeleopCore
- Hardware
    - Leap Motion Controller (through LeapInterface)
    - Robot Arm (through MoteusArmController)
    - Robot Hand (through HandServoController)

Inputs and Outputs:

- Inputs:
    - HumanHandPose (from LeapInterface), containing a decoded representation of the user's hand:
        - 3D positions and orientations of the palm and wrist
        - Per-finger joint angles for the human hand
        - How far are the fingers fanned out
        - Handedness, tracking confidence, and validity flags
    - Configuration Parameters
        - Which human finger maps to which robot finger index i
        - Joint angle ranges for normalization:
            - Human_prox_min[i], human_prox_max[i]
            - Human_mid_min[i], human_mid_max[i]
        - Yaw / spread ranges:
            - Human_yaw_min[i], human_yaw_max[i]
    - TeleopControlInput (from keyboard/UI)
        - Mode selection:
            - mode = TELEOP – live teleoperation (default)
            - mode = MIMIC_RECORD – record a short demonstration
            - mode = MIMIC_PLAYBACK – play back the last recording
        - Keyboard bindings:
            - T – switch to Teleop mode
            - R – start/stop recording (Mimic record)
            - P – start playback of last recording
            - C – start recalibrate the coordinate
            - Esc – abort playback and return to Teleop/Idle
- Outputs:
    - RobotArmCommand – target arm configuration
        - target_joint_positions[5] – desired joint angles for the 5-DOF arm

        - target_end_effector_pose – desired position/orientation of the wrist/end effector
        - These commands are to be used by MoteusArmController
    - RobotHandCommand – target hand configuration
        - Prox_flex_targets[i] – normalized [0,1] flexion of the proximal joint
        - Mid_flex_targets[i] – normalized [0,1] flexion of the middle joint
        - Yaw_targets[i] – normalized [0,1] side-to-side rotation of the proximal bone where 0.0 is fully left, 0.5 is neutral, 1.0 is fully right
        - These commands are to be used by HandServoController.
    - User Interface / Visualization
        - Real-time on-screen interface that informs the operator how their hand is being interpreted and how the robot is responding.

<u>User Interface / Visualization (What the Operator Sees)</u>

1. UltraLeap Virtual Hand
    a. A 3D virtual hand mirrors the operator's tracked hand pose, showing:
        - Finger joint motion (proximal, middle, yaw)
        - Overall hand gesture (open, pinch, grasp)
        - Palm position and orientation
        - Tracking quality (visual fade when confidence is low)
2. Tracking & Confidence Feedback
    a. A status overlay shows:
        - Whether a hand is currently tracked, partially tracked, or lost
        - Leap Motion confidence level
        - Warnings when tracking quality may degrade robot performance
3. Robot State Visualization (Arm + Hand)
    a. A simplified robot model displays:
        - Current robot arm joint angles and end-effector pose
        - Current robot finger joint states (prox flex, mid flex, yaw)
        - Color-coded safety ranges:
            - Green: normal
            - Yellow: near limit
            - Red: limit reached or constrained
4. Latency Indicator
    a. Telemetry indicators display:
        - Round-trip command latency
        - Control loop frequency (Hz)
        - Communication quality (green/yellow/red)
5. Active Joint Highlights

    a. Highlights activate when the operator moves the corresponding human finger joint, improving intuitiveness of the teleoperation mapping
    b. Visual bars or indicators reflect robot command targets:
        - prox_flex_targets[i]
        - mid_flex_targets[i]
        - Yaw_targets[i]
6. Mode Indicator (Teleop vs Mimic)
    a. A mode banner clearly displays the current operating mode to ensure the operator always knows how their hand input is being used
        - Teleop Mode – "Live control. The robot mirrors your hand."
        - Mimic Record – "Recording demonstration. Press R to stop."
        - Mimic Playback – "Replaying stored trajectory. Hand input ignored."
        - Idle/Safe Mode – "Tracking lost or paused. Robot frozen."
7. Safety, Fault, & Recovery Panel
    a. A small system status panel shows:
        - Motor/servo faults
        - Joint limit warnings
        - Communication or tracking failures
        - Recovery steps (e.g., "Waiting for tracking," "Returning to safe posture")
        - Emergency-stop status
    b. This assists operators in monitoring system health

<u>Mapping Design</u>

TeleopCore defines how human hand motion is interpreted:

1. Finger mapping from human to robot hand
    a. Each human finger is mapped to a corresponding robot finger index i
    b. From HumanHandPose, TeleopCore extracts for each finger:
        i. human_prox_flex[i] – proximal flexion angle
        ii. human_mid_flex[i] – middle flexion angle
        iii. human_yaw[i] – side-to-side/abduction angle or spread relative to the palm
    c. These human angles are normalized into the RobotHandCommand fields:
        i. prox_flex_targets[i] – normalized proximal flexion
        ii. mid_flex_targets[i] – normalized middle flexion
        iii. yaw_targets[i] – normalized side-to-side base rotation
    d. All targets are dimensionless values in [0, 1]:
        i. prox_flex_targets[i] = 0 → proximal joint fully extended,
        ii. prox_flex_targets[i] = 1 → fully flexed
        iii. mid_flex_targets[i] = 0 → middle joint fully extended,

iv. mid_flex_targets[i] = 1 → fully flexed
v. yaw_targets[i] = 0 → finger base fully left,
vi. yaw_targets[i] = 0.5 → neutral,
vii. yaw_targets[i] = 1 → fully right

e. TeleopCore applies clamping/smoothing so small hand movements don't cause unstable robot finger motion

2. Hand/Arm mapping from human to robot arm
    a. The human palm position is mapped to a desired end-effector position for the robot arm
    b. TeleopCore uses inverse kinematics from the desired end-effector position to joint angles
    c. The result is a set of the five desired joint angles in target_joint_positions[5] in RobotArmCommand
3. Scaling and workspace limits
    a. Human motion is scaled to fit the robot's reachable workspace
    b. TeleopCore enforces limits so that the commanded joint angles stay within safe joint bounds, even if the human hand moves outside the workspace of the robot arm

<u>Modes and Safety Behavior</u>

TeleopCore supports multiple operating modes that determine how HumanHandPose is used:

- Teleop Mode (live control)
    - Default mode
    - Uses the latest HumanHandPose to continuously compute and stream:
        - RobotArmCommand (arm pose / joint targets)
        - RobotHandCommand (per-finger joint + yaw targets)
    - Sub-modes:
        - Arm and hand
        - Hand-only (arm fixed)
        - Arm-only (for testing)
- Mimic Mode – Record
    - When the user presses the "record" key (R):
        - TeleopCore enters MIMIC_RECORD state.
        - It records a time-stamped sequence of either:
            - HumanHandPose frames, or
            - the corresponding RobotArmCommand + RobotHandCommand at each control step.
    - When the user presses R again:
        - TeleopCore stops recording and stores the trajectory as LastMimicTrajectory.

- Mimic Mode – Playback
    - When the user presses the playback key (e.g., P):
        - TeleopCore enters MIMIC_PLAYBACK state.
        - Hand tracking is paused/ignored; new HumanHandPose frames are not used to compute commands.
        - TeleopCore replays LastMimicTrajectory step by step, regenerating:
            - RobotArmCommand and RobotHandCommand at the original timing
            - These are sent to MoteusArmController and HandServoController instead of live teleop commands.
    - Playback stops when:
        - The trajectory reaches the end, or
        - The user presses Esc or switches back to Teleop mode.
- Idle / Safe Mode
    - If tracking is lost, confidence drops, or the user explicitly pauses:
        - TeleopCore freezes RobotArmCommand and RobotHandCommand, and both move toward a defined safe posture.
    - Idle mode can be entered from Teleop or Mimic at any time if safety conditions are violated.

Mimic Trajectory Representation:

- MimicSample:
    - timestamp – time relative to start of recording
    - RobotArmCommand – desired arm state at that time
    - RobotHandCommand – desired hand state at that time
- MimicTrajectory:
    - samples[] – ordered list of MimicSample
    - duration – total length of the recording (seconds)

TeleopCore maintains a single LastMimicTrajectory in memory for the most recent recording.

Control Flow (High-Level):

At runtime, the TeleopCore module:

1. Read TeleopControlInput
    - Check current mode (TELEOP, MIMIC_RECORD, MIMIC_PLAYBACK, IDLE) and any key presses requesting mode changes.
2. Mode-dependent Command Generation
    - If mode = TELEOP (live teleop)
        - Receive latest HumanHandPose from LeapInterface.
        - Compute robot hand targets:

                - For each finger i, extract human joint/spread angles and normalize to:
                    a. Prox_flex_targets[i]
                    b. Mid_flex_targets[i]
                    c. Yaw_targets[i]
                - Fill RobotHandCommand
            - Compute robot arm targets:
                - Compute desired end-effector pose from palm_position and palm_orientation
                - Run IK → target_joint_positions[5]
                - Fill RobotArmCommand.
            - Apply safety checks and workspace limits
            - Publish RobotArmCommand to MoteusArmController and RobotHandCommand to HandServoController.
            - If the user presses R, switch to MIMIC_RECORD and start a new empty trajectory.
    - If mode = MIMIC_RECORD (recording)
        - Continue to receive HumanHandPose and compute RobotArmCommand + RobotHandCommand as above.
        - At each time step:
            - Add a MimicSample with current timestamp and commands to CurrentMimicTrajectory
        - If user presses R again:
            - Stop recording, store CurrentMimicTrajectory as LastMimicTrajectory
            - Switch to Idle
    - If mode = MIMIC_PLAYBACK (replay)
        - Ignore/skip HumanHandPose for control
        - Use the current playback time to find the appropriate MimicSample.
        - Set RobotArmCommand and RobotHandCommand from the stored sample
        - Send commands to MoteusArmController and HandServoController.
        - Advance playback time until the end of LastMimicTrajectory.
        - If trajectory completes or user presses Esc/T, stop playback and switch to TELEOP/IDLE.
    - If mode = IDLE
        - Freeze or slowly move to a safe pose
        - Do not change commands based on new HumanHandPose
        - Return to TELEOP or MIMIC_PLAYBACK only when requested by TeleopControlInput

3. Update User Interface / Visualization
    - On each cycle, TeleopCore updates the UI components using the latest HumanHandPose, RobotArmCommand, RobotHandCommand, ArmState, and HandState:
    - UltraLeap virtual hand
        - Draw the 3D virtual hand using HumanHandPose.
        - Portray visibility/brightness based on tracking confidence.
    - Tracking & confidence indicators
        - Show “Tracked / Partially Tracked / Lost” + confidence bar.
    - Robot state visualization
        - Render current robot arm pose and finger joint states.
        - Color-code joints by proximity to limits (green/yellow/red).
    - Command and responsiveness indicators
        - Display mode (Teleop, Mimic Record, Mimic Playback, Idle).
        - Show latency / control-loop rate and connection status.
        - Highlight active joint targets (prox_flex_targets, mid_flex_targets, yaw_targets) with bars/indicators.
    - Safety, fault, and recovery panel
        - Display any fault flags (limit reached, E-stop, comms timeout).
        - Show recovery messages (“Waiting for tracking”, “Returning to safe posture”).
4. Provide feedback to other modules
    - TeleopCore forwards the current mode, RobotArmCommand, RobotHandCommand, and robot state to:
        - HapticsController (for rendering glove feedback).
        - System monitor / logging tools (for debugging and later analysis).

Robot Arm Module (MoteusArmController):

Purpose: The Robot Arm module controls a 5-DOF robotic arm used to position the robot hand in space.

The module receives desired arm motion derived from the human hand motion and computes joint-level commands to move the robot arm end effector accordingly.

External Dependencies:

- Hardware:
    - Custom 5-DOF robotic arm (Dr. Bot arm)
    - 5x Steadywin BLDC motors
    - 5x Moteus-r4 Motor Drivers
    - mjbots mjcanfd-usb-1x USB-CAN FD adapter

- Software:
    - Moteus motor driver library (Python / C++)
    - tview (for debugging and visualization during development)
    - LeapInterface Module
    - TeleopCore Module
    - MoteusArmController Module

Inputs and Outputs:

- Inputs:
    - RobotArmCommand (from TeleopCore)
        - Target joint angles or target end effector pose
- Outputs:
    - Joint position commands sent to the arm motors
    - ArmState feedback
        - Joint positions
        - Joint velocities
        - Driver status/fault flags

Arm Hardware and Control Interface:

The robot arm has five joints, each actuated by a BLDC motor controlled by a moteus-r4 motor driver.

Each moteus driver:

- Implements Field-Oriented Control (FOC)
- Includes a built-in encoder for joint position feedback
- Supports PID control at the motor level

All moteus drivers are daisy-chained on a CAN FD bus, which connects to the laptop via the mjcanfd-usb-1x adapter. High-level control is performed on the laptop using the moteus Python library.

Each control cycle:

- Desired joint angles are sent to the corresponding moteus drivers
- Encoder feedback is returned to software for state estimation and monitoring

Control Flow (High-Level):

At runtime, the module:

1. Initializes communication with all moteus motor drivers over CAN FD
2. Receives a RobotArmCommand from the TeleopCore module
3. If human hand position changes, computes inverse kinematics to get joint angles from given end effector position
4. Sends join position commands to each moteus driver using moteus API

5. Receives joint state information from the drivers
6. Publishes the ArmState for logging and monitoring

Robot Hand Module (HandServoController)

Purpose: The Robot Hand Module is responsible for actuating the tendon-driven robot hand based on the finger targets computed by the TeleopCore.

Each finger has three actuated DOF:

1. Proximal flexion – bending the proximal finger bone forward/backward
2. Middle flexion – bending the middle finger bone forward/backward
3. Proximal yaw – moving the proximal bone side-to-side at the base

Mechanically, each finger consists of three links with torsion springs at the joints. The springs bias the joints toward a closed configuration, and cable routing is used to pull the links open against the springs. The distal link is passive with no motor. At the base, each finger assembly is mounted on a vertical pivot directly driven by a servo for side-to-side motion.

The HandServoController converts finger targets from TeleopCore into servo commands for:

- Proximal flexion motor
- Middle flexion motor
- Proximal yaw (side-to-side) motor

for each finger.

External Dependencies:

- Hardware:
    - Custom robot hand with:
        - 3 links per finger with torsion springs at joints
        - Distal link with hardstops and no motor
        - Cable routing to open joints against the springs
        - 3 motors per finger:
            - 1 proximal flexion motor
            - 1 middle flexion motor
            - 1 proximal yaw motor
    - Teensy microcontroller driving the finger motors
- Software:
    - Arduino IDE with Teensyduino addon
    - LeapInterface Module

- TeleopCore Module
- HandServoController Module

Inputs and Outputs:

- Input
  - RobotHandCommand from TeleopCore containing, for each finger i:
    - Prox_flex_targets[i] – target for proximal flexion joint where
      - 0.0 is fully extended
      - 1.0 is fully flexed
    - Mid_flex_targets[i] – target for middle flexion joint
      - 0.0 is fully extended
      - 1.0 is fully flexed
    - Yaw_targets[i] – targets for side-to-side base rotation
      - 0.0 is fully left
      - 0.5 is neutral
      - 1.0 is fully right
- Outputs
  - Per-finger motor commands:
    - Prox_flex_angle[i] – angle for proximal joint motor
    - Mid_flex_angle[i] – angle for middle joint motor
    - Prox_yaw_angle[i] – angle for proximal yaw servo
  - HandState
    - Last commanded angle per joint and yaw
    - Sensor feedback and status flags like limits hit or faults

Mapping Design

HandServoController maps the normalized targets [0,1] into actual servo angles using per-finger calibration.

1. Proximal Flexion Mapping
   a. For each finger i, calibration constants are defined
      i. Prox_min_angle[i] – servo angle where the proximal joint is fully extended
      ii. Prox_max_angle[i] – servo angle where the proximal joint is fully flexed
      Then: prox_flex_angle[i] = prox_min_angle[i] + prox_flex_targets[i] * (prox_max_angle[i] - prox_min_angle[i])
      iii. Prox_flex_targets[i] = 0.0 which is the joint at prox_min_angle[i] (extended)
      iv. Prox_flex_targets[i] = 1.0 which is the joint at prox_max_angle[i] (flexed)
2. Middle Flexion Mapping
   a. Similarly, define:

i. Mid_min_angle[i] – angle for fully extended middle joint
ii. Mid_max_angle[i] – angle for fully flexed middle joint
iii. Then: mid_flex_angle[i] = mid_min_angle[i] + mid_flex_targets[i] * (mid_max_angle[i] - mid_min_angle[i])

3. Proximal Yaw Mapping
    a. Yaw_left_angle[i] – mechanical left limit
    b. Yaw_right_angle[i] – mechanical right limit
    c. Then, prox_yaw_angle[i] = yaw_left_angle[i] + yaw_targets[i] * (yaw_right_angle[i] - yaw_left_angle[i])
    d. Yaw_targets[i] = 0.0 is fully left
    e. Yaw_targets[i] = 0.5 is neutral
    f. Yaw_targets[i] = 1.0 is fully right

Control Flow (High Level)

At runtime, the HandServoController module:

1. Receives the latest RobotHandCommand from TeleopCore
2. For each finger i:
    a. Reads prox_flex_targets[i], mid_flex_targets[i], and prox_yaw_targets[i]
    b. Computes prox_flex_angle[i], mid_flex_angle[i], and prox_yaw_angle[i] using calibrated linear mappings
3. Packs motor targets for all fingers into a serial command frame
4. Sends the frame over USB serial to the Teensy
5. The Teensy firmware:
    a. Updates PWM outputs for all three motors per finger
    b. Sends feedback including estimated angle and error flags as HandState
6. HandServoController updates and publishes HandState for:
    a. HapticsController for glove feedback
    b. System monitoring and visualization

Haptic Feedback & Glove Module (HapticsController + GloveInterface)

Purpose:

The Haptic Feedback & Glove Module is responsible for rendering force feedback to the human operator through a LucidVR-style cable-driven glove. It adjusts cable tension on the fingers so the user feels resistance when the robot hand grasps objects, contacts the environment, or reaches workspace or joint limits.

External Dependencies:

- Hardware:

        - LucidVR-style glove with cable-driven force feedback, including:
            - Small motors that pull the cables routed to the fingers
            - Cables that resist finger motion by increasing tension
            - Arduino Nano ESP32 mounted to glove
                - Bluetooth connection between Arduino and laptop
- Software:
    - Arduino IDE
    - GloveInterface
        - handles communication between laptop and Arduino
    - HapticsController Module
        - Sends haptic commands to motors for force feedback

Inputs and Outputs:

- Inputs (to HapticsController):
    - ArmState (from MoteusArmController) with robot arm joint positions
    - HandState (from HandServoController) with robot finger positons
- Outputs:
    - HapticCommand (from HapticsController to GloveInterface) with:
        - haptic_level[5] – per-finger cable tension commands where
            - 0 = no resistance
            - 1 = max allowable cable tension, strong resistance

HapticsController Design

The HapticsController determines how much force feedback should be applied to the user based on robot motion and interaction

- Cable tension is increased[less freedom to move] when:
    - The robot hand closes around an object
    - The robot hand contacts the environment
    - The robot arm or hand reaches a workspace or joint limit
- Cable tension is reduced[more freedom to move] when:
    - The robot hand is open
    - No contact or constraint is present

GloveInterface Design:

The GloveInterface handles communication between the laptop and Arduino

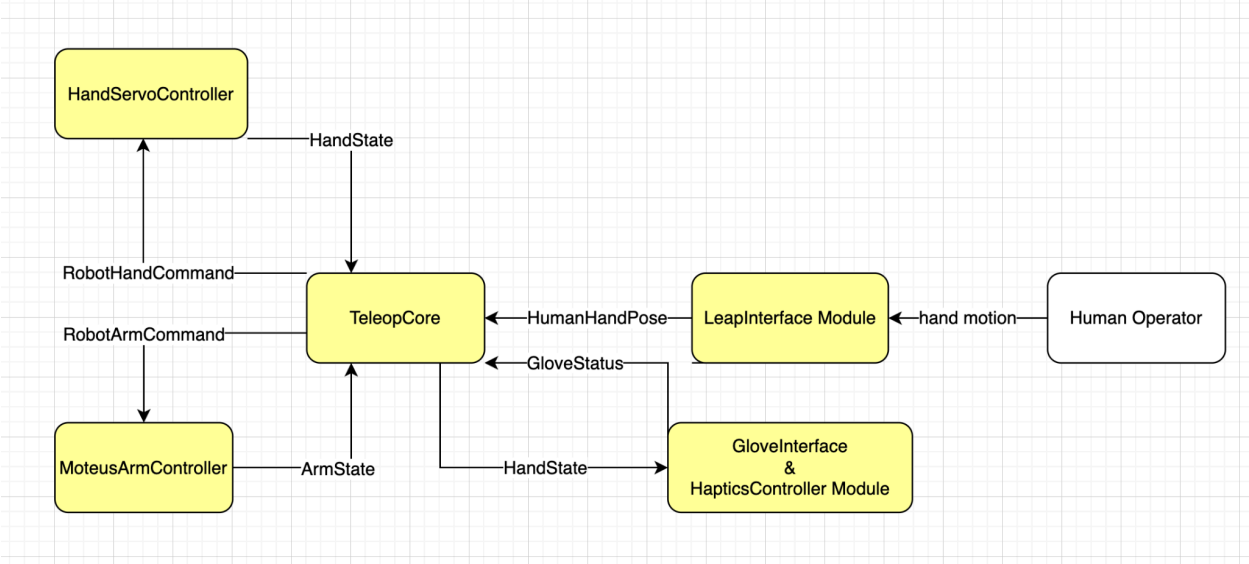
- Opens and maintains bluetooth serial connection to the glove
- Stores HapticCommand data
- Sends per-finger cable-tension commands to Arduino

Control Flow (High-Level):

At runtime, the Haptic Feedback & Glove Modules function as follows:

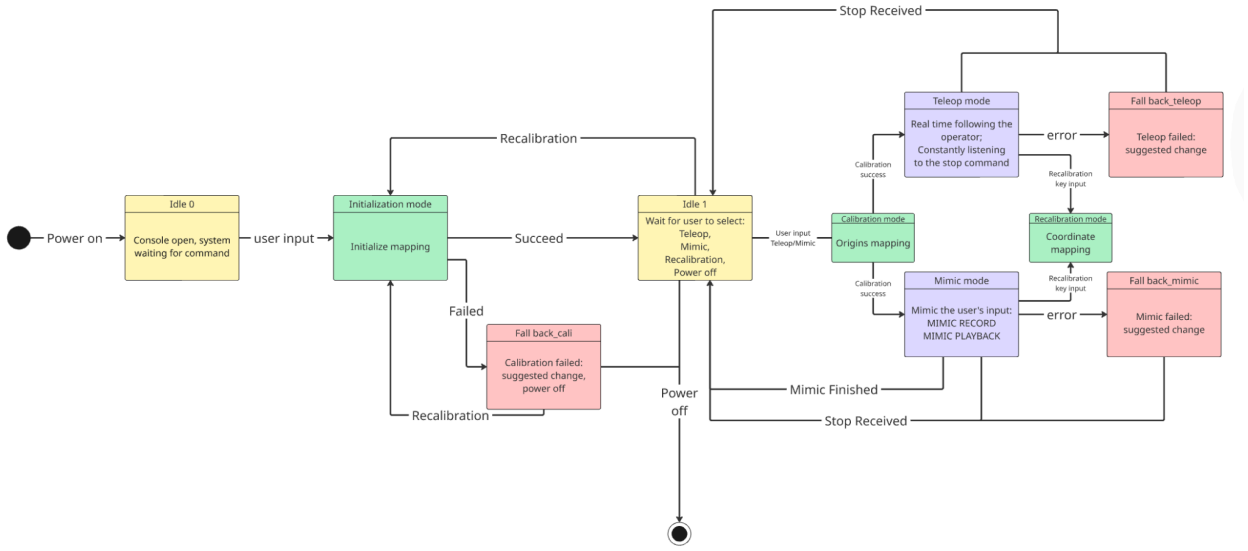
1. State collection
    a. HapticsController receives ArmState and HandState from the robot arm and hand modules
2. Haptic computation
    a. Computes per-finger cable tension commands (haptic_level[ ]) based on robot motion and contact
3. Command communication
    a. Sends a HapticCommand to the GloveInterface
4. Glove actuation
    a. GloveInterface sends commands to the Arduino
    b. Arduino adjusts motor outputs to apply the specified cable tension to each finger
5. Status data returned
    a. The glove's status data is received by GloveInterface to be sent to system monitoring

Full Software Diagram:

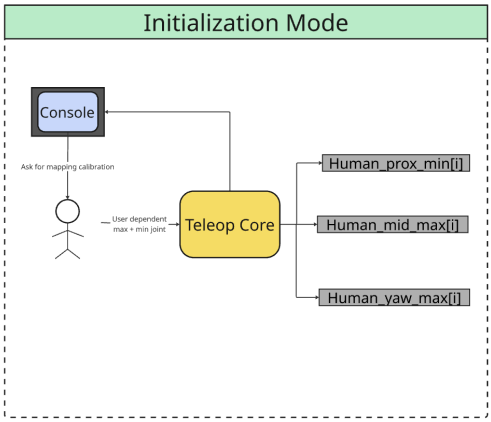


8. Statecharts:

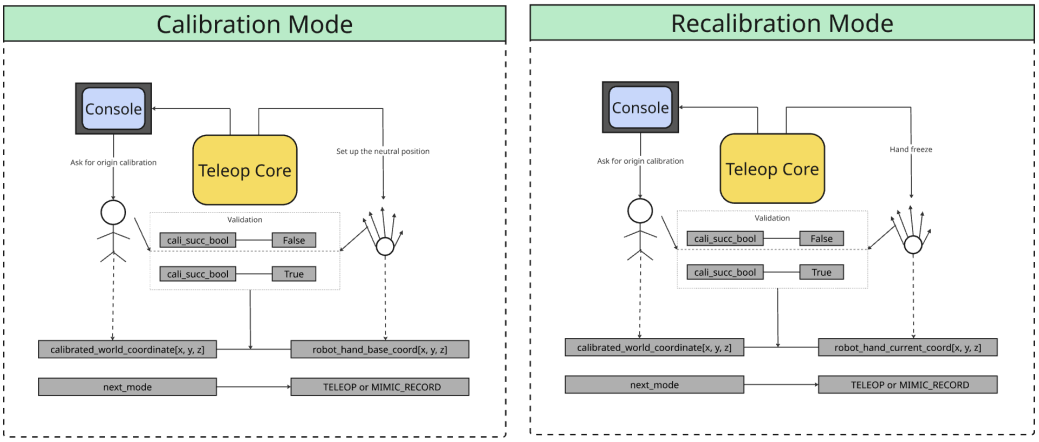
Overall System State chart



Operation sequence: As the user powers on the robotic system, the console will open and the system will enter the Idle 0 mode waiting for the user input to start initialization. As the user makes a keyboard input, the system will enter the initialization mode that maps out the user hand limit to the robot hand limit(as described below). Once the initialization succeeds, the system will enter the Idle 1 mode in which the console lists out the modes the user can choose from. When the user inputted a command to enter the real time/mimic mode. The system will enter the calibration mode that calibrates the user's coordinate origin to the robot hand coordinate origin. Once the calibration succeeds, the system will enter the corresponding mode as entered by the user. For the two modes, the user can enter keyboard commands to stop the mode or recalibrate their coordinates. There are fall back modes for every important mode of operation, and once the fall back is resolved, the system will jump back to idle 1 mode.



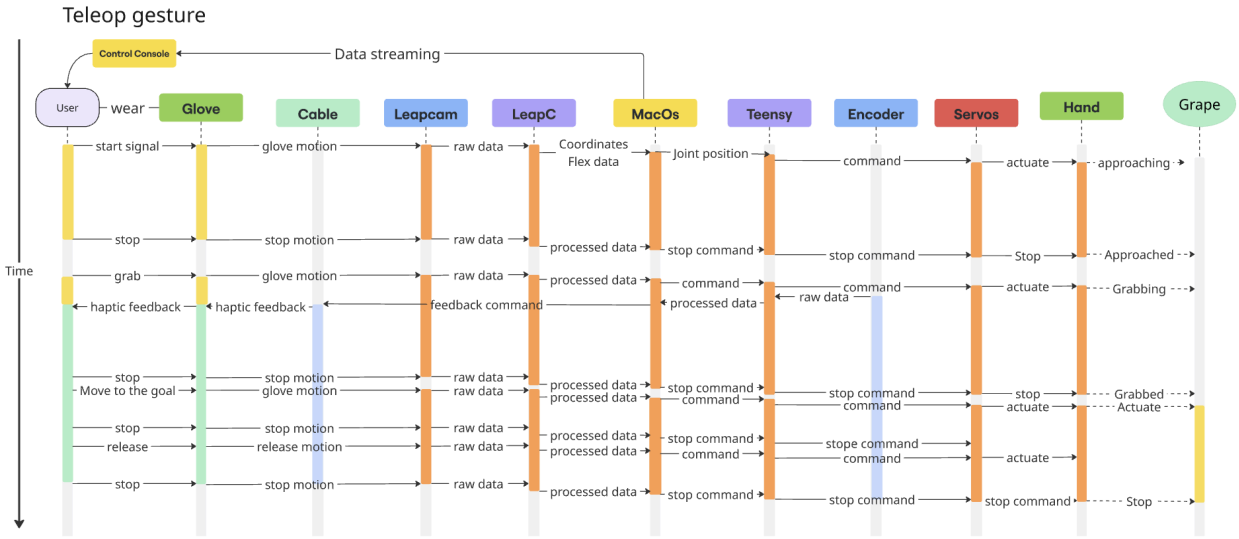
Initialization mode: Console asks the user to calibrate their maximum/minimum joint extending positions. Processed by LeapC, the information will be recorded to the variables.

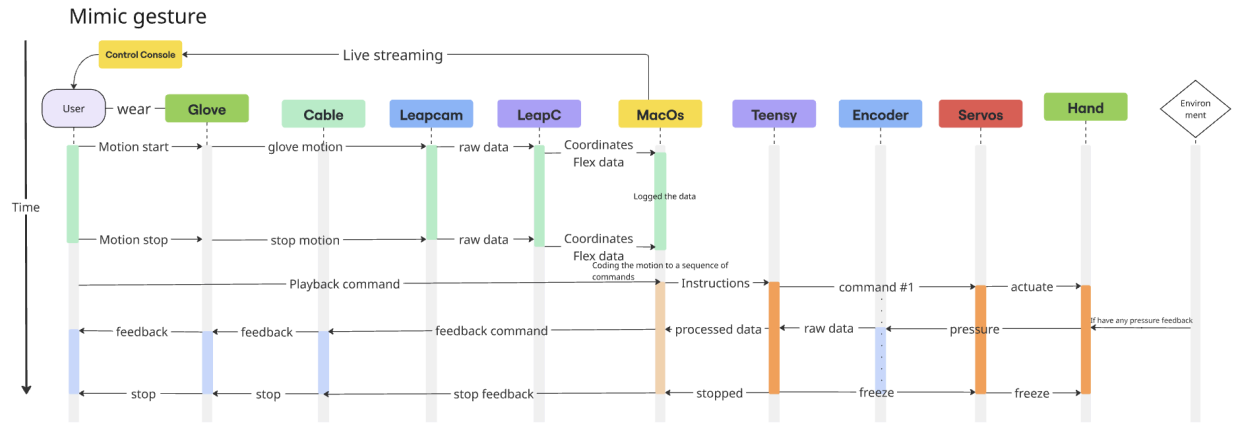


Calibration mode: The first calibration the system needs to do when entering the Teleop/mimic mode. The consoles ask the user to place their hand to an origin that they feel comfortable to operate on. Once the mapping gets validated, the origin(calibrated_world_coordinate[x, y, z]) will represent the neutral position(robot_hand_base_coord[x, y, z]) of the robot hand.

Recalibration mode: The users can pause the operation mode and recalibrate the base coordinate for better maneuvers. The robot will freeze to the last position, and the position the user recalibrated will be mapped to the robot's last position.

Sequence diagrams for the two operation modes:





9. Fault Recovery:

Fault Recovery and Degraded Modes – Electrical Protection

The electrical design intentionally separates the logic rails from the actuator rail so that faults on high-current lines do not immediately shut down control electronics. The 22 V supply from the wall first passes through a fuse and protection module, then is stepped down to 5 V. From this 5 V bus we derive two paths: a 5 V logic rail that powers the Teensy and low-power sensors, and a separate 5 V servo rail (through the E-stop) that powers only the hand servos and current monitors. The arm's BLDC drivers remain on the 22 V actuator bus. All modules share a common ground.

Shorts or over-current on the 5 V servo rail are contained by the fuse/polyfuse and the buck converter current limit. In this case the servos and digital power monitors will lose power, but the 5 V logic rail and Teensy remain active. The degraded mode is "arm and sensing only": the arm can still hold or move to a safe pose under software control, while the Teensy reports the fault (e.g., loss of servo bus voltage or unexpected current readings) and ignores further servo commands until the fault is cleared and the supply is reset or the fuse is replaced.

If a single servo or finger group draws abnormal current without a hard short, the digital power monitors can detect the sustained over-current and the firmware can selectively disable that finger by holding its PWM command at a relaxed position. In this degraded mode the hand continues operating with reduced capability (one or more fingers disabled), but the rest of the system remains powered and controllable. This limits thermal stress on the affected servo and wiring while still allowing teleoperation and data collection.

If the E-stop is pressed, the 5 V servo rail and the 22 V actuator bus are both interrupted, removing power from all hand servos and arm motors. The Teensy, FSR dividers, and measurement electronics stay powered from the 5 V logic rail, and the communication link to the

computer remains active. The system enters a safe “no-actuation” mode: all joints are unpowered, but the controller continues to monitor the E-stop sense input and bus voltages. Recovery consists of clearing the physical hazard, releasing the E-stop, and then re-enabling arm/hand motion in software.

If the 22 V supply or buck converter fails (e.g., over-temperature shutdown or upstream fuse blow), both the arm and the hand lose actuator power but logic remains powered as long as the 5 V logic rail is intact. The robot transitions to a passive state where it cannot move but can still log, communicate, and display diagnostics. Recovery in this case requires identifying and correcting the root cause (e.g., mechanical jam leading to high current) and then replacing or resetting the affected power component.

Finally, sensor-side faults (open-circuit or shorted FSRs, failed current monitor) affect only state estimation for that finger or group. The controller will treat missing or saturated readings as a sensor fault, fall back to open-loop position control for that finger, and continue operating the remaining sensors normally. This keeps the electrical fault surface localized: a single failed fingertip sensor or monitor results in reduced feedback for that digit, not a full system shutdown.

Fault Recovery and Degraded Modes for Software:
This section explains how the software handles faults and enters degraded modes while attempting to maintain functionality wherever possible.

<u>Loss of Hand Tracking (LeapInterface / TeleopCore)</u>
Fault Condition

- No valid HumanHandPose received from LeapInterface within a given period of time
- Leap reports no hands, or the tracking quality drops below a certain threshold

Detection

- TeleopCore monitors timestamps and validity flags on HumanHandPose

Recovery / Degraded Mode

- TeleopCore enters Idle Teleop Mode:
    - RobotArmCommand holds current joint positions
    - RobotHandCommand gradually opens fingers to a neutral safe pose
    - HapticsController sets all haptic_level values to 0 and releases any tension
    - When valid tracking resumes, TeleopCore can re-enable teleoperation

<u>Robot Arm Faults (MoteusArmController)</u>
Fault Condition

- Moteus driver reports an error status
- Loss of communication with one or more joints

- Measured joint positions are very different from commanded positions

Detection
- MoteusArmController reads status flags and joint state from each driver
- TeleopCore checks for communication timeouts or abnormal status codes

Recovery / Degraded Mode
- MoteusArmController:
    - Immediately stops sending new motion commands
    - Commands a safe stop or hold current joint positions, depending on fault type
- TeleopCore:
    - Switches to Arm-Disabled Mode where:
        - RobotArmCommand is frozen
        - RobotHandCommand can remain active for testing, if possible
- HapticsController:
    - Can optionally reduce tension or disable haptic feedback to indicate that arm motion is no longer be executed

Robot Hand / Servo Faults (HandServoController)

Fault Condition
- Loss of serial communication with robot hand's Arduino
- Arduino reports an error, or fails to acknowledge commands

Detection
- TeleopCore monitors serial link and acknowledgements
- Timeouts if no response is received within a given period of time

Recovery / Degraded Mode
- TeleopCore:
    - Stops sending new RobotHandCommand updates
    - Enters Hand-Disabled Mode where robot arm teleoperation may continue, but finger commands are ignored
- HandServoController:
    - Moves servos to a neutral "hand open" position
- HapticsController:
    - Reduces any tension and disables haptics related to the hand

Glove / Haptic Faults (GloveInterface / HapticsController)

Fault Condition

- Loss of communication with the Arduino Nano ESP32
- Arduino reports error with motors

Detection

- GloveInterface monitors bluetooth connection and any status messages from the glove
- Timeouts or error codes from the Arduino Nano ESP32

Recovery / Degraded Mode

- HapticsController:
    - Immediately sets all haptic_level values to 0
- GloveInterface:
    - Stops sending new haptic commands until the connection is restored
- Teleoperation (TeleopCore, HandServoController, MoteusArmController):
    - Continues to operate in a Haptics-Off Mode
        - Robot arm and hand motion remain active
        - The user no longer receives force feedback but the rest of the system is usable

General Software Failure / Watchdog Behavior

Fault Condition

- Any software module stops updating at the required rate

Detection

- A simple watchdog timer in the main control loop monitors last-update timestamps for modules

Recovery / Degraded Mode

- On watchdog timeout:
    - Stop sending new robot arm and hand commands (hold or move to neutral)
    - Set all haptic outputs to 0
- Require manual restart or operator intervention before re-enabling full teleoperation

10. Operations Plan:

During deployment, the teleoperation system is set up on a stationary lab bench. The robotic arm is securely mounted to the table, and the arm and hand microcontrollers are connected to the host computer via USB. The Leap Motion Controller is placed on the table with a clear view of both the operator's hand and the work area, and is also connected to the computer over USB. Finally,

the shared power supply for the robotic arm and hand is plugged into a wall outlet to provide stable power to all actuators.

In preparation for operation, the wearable device (glove) is paired and connected to the computer via Bluetooth. The robotic arm and hand are driven to their home positions and placed in an idle state, and the wearable device is also initialized to its idle state. The camera (if used in addition to Leap Motion) is positioned, focused on the workspace, and starts streaming frames into the software. In parallel, the simulation software is launched, configured for the current hardware setup, and left in an idle mode awaiting commands.

At start-up, the robotic arm and hand execute an automatic self-calibration routine to verify joint limits and zero their encoders. The simulation environment is then initialized or calibrated as needed so that the virtual model matches the physical arm configuration. Finally, a synchronization calibration is performed between the glove and the robotic arm/hand, mapping the operator’s hand pose space to the robot’s joint space, so that subsequent motion of the wearable device results in accurate, real-time teleoperation.

11. Usage Guidelines:

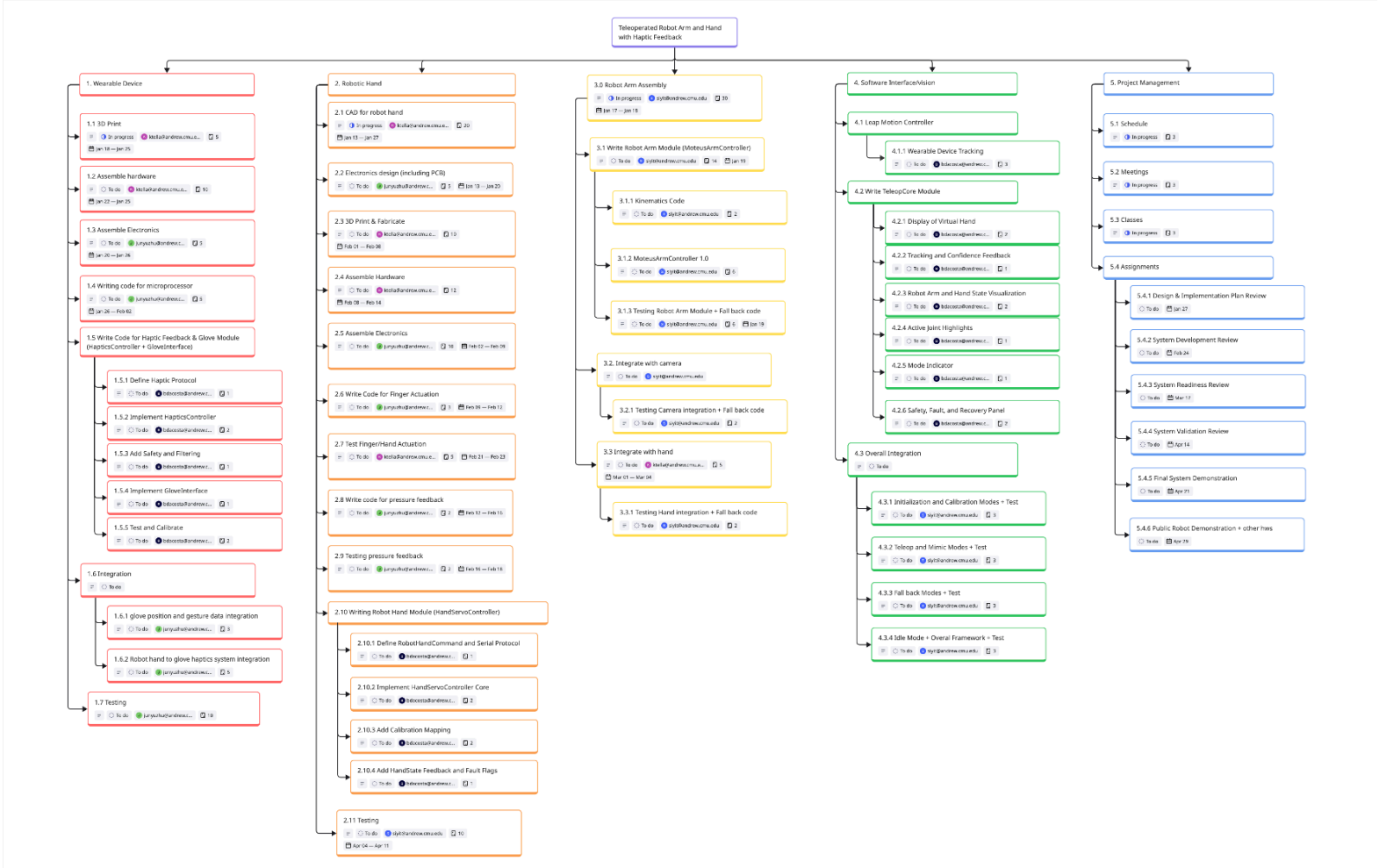
To ensure the reliability of the teleoperation system and the safety of the operator and hardware, specific operational boundaries must be maintained. First and foremost, the operator's hand must remain within the trackable region of the sensors at all times. Specifically, the Leap Motion Controller must have a clear, unobstructed view of the operator's hand to maintain accurate input data it also must not be moving; if the hand exits this trackable volume or tracking is lost, the system may need to enter a safe fallback mode to prevent erratic behavior.

Furthermore, the robotic arm is constrained physically and must only be commanded to move within a reachable range that doesn't have obstacles. The control software maps the operator's motions to the robot's workspace, and care must be taken to ensure the "glove motion map" does not demand positions outside the hand's valid motion space Finally, to prevent mechanical stress and ensure safe interaction, the robot arm is software-limited to operate only within a safe speed range, this ensures that even during weird operator movements, the arm actuators remain within their performance limits and can be safely halted via the E-stop if necessary.

Management Plan

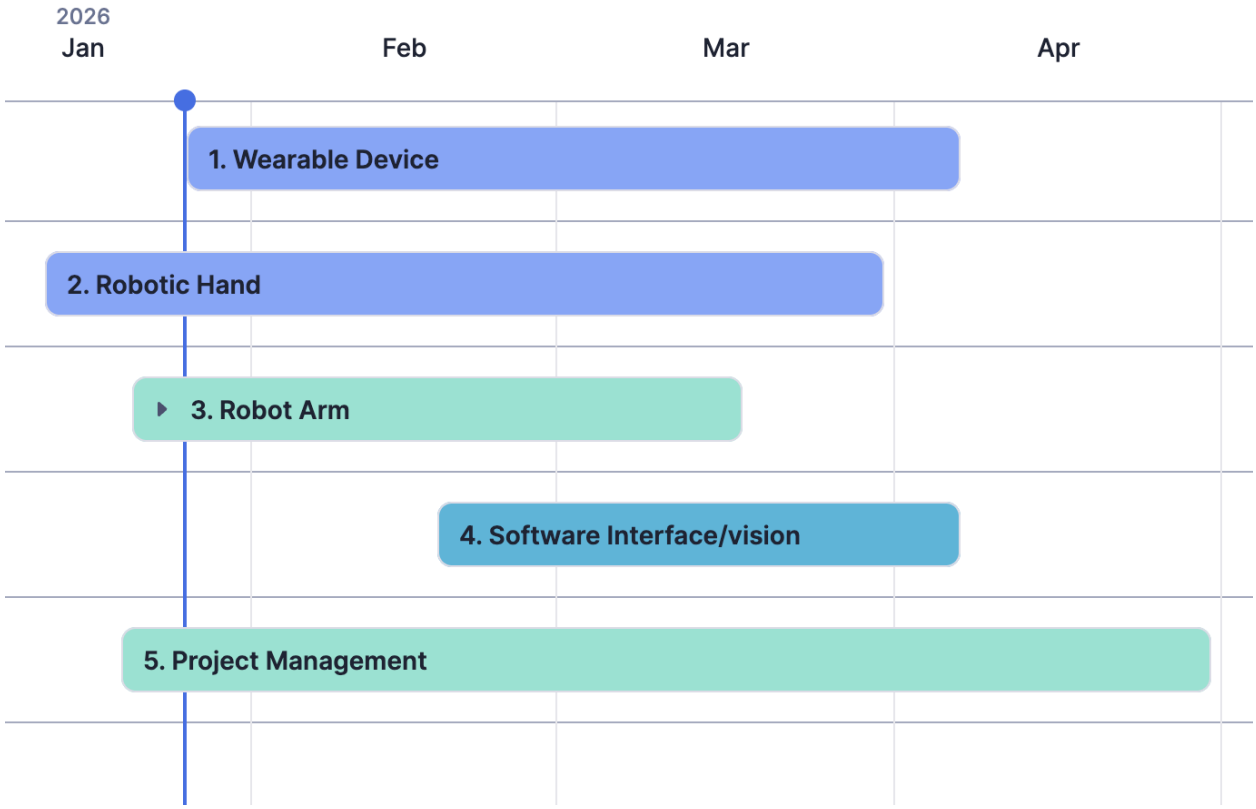
We identify and categorize tasks based on the subsystems of our project: the wearable device, robotic hand, robot arm, and software integration. Each task is then assigned and organized into a timeline managed through smartsheet.com. We have a Miro Work Breakdown Structure of all the tasks to be completed down to 3 levels of detail. The WBS was exported to smartsheet which has a timeline and Gantt chart view of the tasks to be completed.

1. Miro Work Breakdown Structure

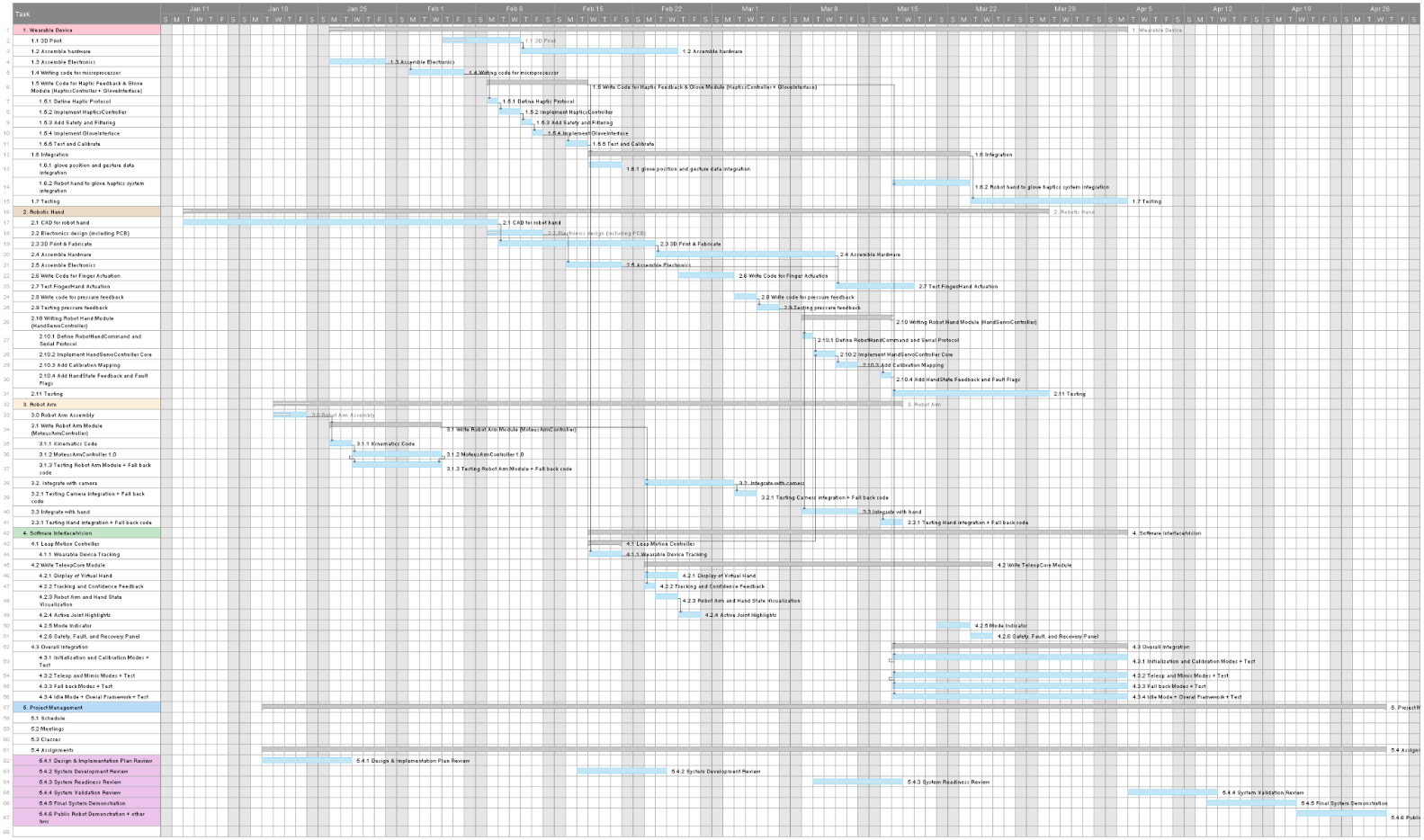


2. Smartsheet Timeline (High-Level Picture)

Timeline containing level 3 detail can be accessed through the linked title.



3. Smartsheet Gantt Chart


Task
Jan 11
Jan 18
Jan 25
Feb 1
Feb 8
Feb 15
Feb 22
Mar 1
Mar 8
Mar 15
Mar 22
Mar 29
Apr 5
Apr 12
Apr 19
Apr 26
1. Wearable Device
1.1 3D Print
1.2 Assemble hardware
1.3 Assemble Electronics
1.4 Writing code for microprocessor
1.5 Write Code for Haptic Feedback & Glove Module (HapticsController + GloveInterface)
1.5.1 Define Haptic Protocol
1.5.2 Implement HapticsController
1.5.3 Add Safety and Filtering
1.5.4 Implement GloveInterface
1.5.5 Test and Calibrate
1.6 Integration
1.6.1 glove position and gesture data integration
1.6.2 Robot hand to glove haptics system integration
1.7 Testing
2. Robotic Hand
2.1 CAD for robot hand
2.2 Electronics design (including PCB)
2.3 3D Print & Fabricate
2.4 Assemble Hardware
2.5 Assemble Electronics
2.6 Write Code for Finger Actuation
2.7 Test Fingers/Hand Actuation
2.8 Write code for pressure feedback
2.9 Testing pressure feedback
2.10 Writing Robot Hand Module (HandSeeroController)
2.10.1 Define RobotHandCommand and Serial Protocol
2.10.2 Implement HandSeeroController Core
2.10.3 Add Calibration Mapping
2.10.4 Add HandState Feedback and Fault Flags
2.11 Testing
3. Robot Arm
3.0 Robot Arm Assembly
3.1 Write Robot Arm Module (MofaccArmController)
3.1.1 Kinematics Code
3.1.2 MofaccArmController 1.0
3.1.3 Testing Robot Arm Module + Fail back code
3.2 Integrate with camera
3.2.1 Testing Camera Integration + Fail back code
3.3 Integrate with hand
3.3.1 Testing Hand integration + Fail back code
4. Software Interface/Vision
4.1 Leap Motion Controller
4.1.1 Wearable Device Tracking
4.2 Write TeleopCore Module
4.2.1 Display of Virtual Hand
4.2.2 Tracking and Confidence Feedback
4.2.3 Robot Arm and Hand State Visualization
4.2.4 Active Joint Highlights
4.2.5 Mode Indicator
4.2.6 Safety, Fault, and Recovery Panel
4.3 Overall Integration
4.3.1 Initialization and Calibration Modes + Test
4.3.2 Teleop and Mimic Modes + Test
4.3.3 Fail back Modes + Test
4.3.4 Idle Mode + Overal Framework + Test
5. Project Management
5.1 Schedule
5.2 Meetings
5.3 Classes
5.4 Assignments
5.4.1 Design & Implementation Plan Review
5.4.2 System Development Review
5.4.3 System Readiness Review
5.4.4 System Validation Review
5.4.5 Final System Demonstration
5.4.6 Public Robot Demonstration + other tbns

References

[1] J. Jin, S. Wang, Z. Zhang, D. Mei, and Y. Wang, “Progress on flexible tactile sensors in robotic applications on objects properties recognition, manipulation and human-machine interactions,” Soft Sci., vol. 3, p. 8, 2023. DOI: 10.20517/ss.2022.34.

[2] A. U. Nisa, S. Samo, R. A. Nizamani, A. Irfan, Z. Anjum, and L. Kumar, “Design and Implementation of Force Sensation and Feedback Systems for Telepresence Robotic Arm”, *J Robot Control (JRC)*, vol. 3, no. 5, pp. 710–715, Sep. 2022.

[3] A. Al-Samarraie, M. Eldegwy, H. M. Yusof, and D. Alzahrani, “A brief survey of telerobotic time delay mitigation,” Frontiers in Robotics and AI, vol. 7, no. 119, pp. 1–15, Oct. 2020. doi: 10.3389/frobt.2020.578805

[4] Lee, M. H., & Nicholls, H. R. (1999). *Tactile sensing using force sensing resistors and a super-resolution algorithm.* IEEE Transactions on Robotics and Automation, 15(4), 557–566. https://doi.org/10.1109/70.772982

[5] Jentoft, L. P., Howe, R. D., & Cutkosky, M. R. (2006). *Miniature force sensor for prosthetic and robotic applications.* NASA Technical Report NASA/TM-2006-26212. https://ntrs.nasa.gov/api/citations/20060026212/downloads/20060026212.pdf

[6] Zhou, J., Yuan, W., & Suo, Z. (2025). *Fluidically innervated lattices make versatile and durable tactile sensors.* arXiv preprint arXiv:2507.21225. https://arxiv.org/abs/2507.21225

[7] Barrett Technology. *BarrettHand BH8-282*. https://barrett.com/barretthand

[8] Shadow Robot Company. *The Shadow Dexterous Hand Series*. https://shadowrobot.com/dexterous-hand-series/

[9] Festo. *BionicSoftHand*. https://www.festo.com/us/en/e/about-festo/research-and-development/bionic-learning-network/bionic-grippers-and-soft-robots/bionicsofthand-id_68106/

[10] Sarcos Robotics. *Guardian XO*.https://robotsguide.com/robots/guardianxo

[11] Apple Inc. *Apple Vision Pro*. https://www.apple.com/apple-vision-pro/

[12] Microsoft. *Azure Kinect DK*. https://azure.microsoft.com/en-us/products/kinect-dk

[13] CyberGlove Systems. *CyberGlove III*. http://www.cyberglovesystems.com/cyberglove-iii

[14] SenseGlove. *SenseGlove Nova 2*. https://www.senseglove.com/

[15] Zhang, H., Hu, S., Yuan, Z., & Xu, H. (2025). *DOGlove: Dexterous manipulation with a low-cost open-source haptic force feedback glove*. arXiv. https://doi.org/10.48550/arXiv.2502.07730

[16] Lucas VRTech. (n.d.). *LucidGloves: Open source VR haptic gloves* (Version 3.1) [Hardware and software documentation]. GitHub. https://github.com/LucidVR/lucidgloves

[17] Ultraleap. (2013). *Leap Motion Controller* [Hardware]. https://www.ultraleap.com/product/leap-motion-controller/

[19] Logitech. (2012). *C920 HD Pro Webcam* [Hardware]. https://www.logitech.com/en-us/products/webcams/c920-pro-hd-webcam.html

[20] Precision Microdrives. (n.d.). *Linear Resonant Actuators (LRA): Application Note AB-020*. https://www.precisionmicrodrives.com/vibration-motors/linear-resonant-actuators-lras/